

**Deployment and Optimization of Large
Language Models and Retrieval-Augmented
Generation Pipelines on High-Performance
Computing Systems**

Bachelor of Science in Computer Science

Author: Nikos Kalogirou

Supervisor: Dr. Christodoulos Stylianou

Declaration

I hereby declare that this thesis is my own work and has not been submitted in any form for another degree or qualification at any university or institution.

Acknowledgements

I would like to thank Dr. Christodoulos Stylianou, my supervisor, for guiding, giving constructive feedback and supporting me in the development of this project. His guidance enabled me to focus the work, enhance the technical aspects of implementation, and structure my thesis. Finally, I want to express my gratitude to the Department of Computer Science and Engineering at European University Cyprus for the academic environment and the resources it offered in the course of this work.

I would also like to thank the Department of Computer Science and Engineering at European University Cyprus for providing the academic environment and resources that supported this work. I am grateful to the lecturers and staff who contributed to my studies and helped me build the knowledge needed to complete this project.

Finally, I would like to thank my family and friends for their encouragement, patience, and support during the preparation of this thesis. Their motivation and understanding were important throughout the research, implementation, and writing process.

Abstract

Large Language Models (LLMs) are now playing a crucial role in various applications of Natural Language Processing (NLP), such as question answering, summarization, and information retrieval. However, the high computational and operational cost of inference, document retrieval, indexing and data handling should be taken into account when designing a system. One way to address this issue is by combining a language model with an external source of knowledge, which can then be used to retrieve relevant documents and “feeding” them to the language model as a context before generating an answer: Retrieval-Augmented Generation (RAG).

This thesis explores deployment and evaluation of RAG pipelines, particularly in terms of retrieval-level optimization, reproducibility and deployment portability. The experimental design is staged. The first phase consists of six pipelines implemented and evaluated locally on a personal computer: an in-memory pipeline as a baseline, a Milvus vector-database pipeline with an additional tuned one, a dense retrieval pipeline with reranking, a hybrid retrieval pipeline, and a hybrid retrieval pipeline with reranking. In the second stage, the same implementations, supporting files, scenario queries and result-logging structure will be run on a High-Performance Computing (HPC) environment. This enables the comparison between the variants of the RAG in each of them.

The evaluation framework takes into account the performance of the system as well as the quality of the output. The system level measurements are the following: *index time*, *retrieval time*, *generation time*, and *total execution time*. The evaluation of output-quality takes into account aspects such as the relevance of the retrieved documents, the correctness of the answer, the groundedness, the completeness, and the risk of hallucination. This is relevant as the speed with which a RAG system returns information is not necessarily its most important characteristic, it should also be able to provide useful evidence to enable the generation of responses which are backed by evidence. The thesis provides a structured local comparison of the implemented RAG variants, it provides a reproducible workflow including a run and log experiment, and it provides a framework for extending the same files to running experiments on HPC. Local deployment and comparison are the baseline at this stage. The next experimental stage is the implementation

of the HPC and a comparison between local and HPC; this will be completed in the results chapter with the appropriate measurements. Full model training is not included in the main, as is full model fine-tuning and large-scale multi-node inference experimental scope.

List of Abbreviations

NLP	Natural Language Processing
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
HPC	High-Performance Computing
GPU	Graphics Processing Unit
CPU	Central Processing Unit
KV	Key-Value
DPR	Dense Passage Retrieval
BM25	Best Matching 25
HNSW	Hierarchical Navigable Small World
FAISS	Facebook AI Similarity Search
TP	Tensor Parallelism
PP	Pipeline Parallelism
DP	Data Parallelism
FSDP	Fully Sharded Data Parallel
TGI	Text Generation Inference
TIS	Triton Inference Server
MMLU	Massive Multitask Language Understanding
HELM	Holistic Evaluation of Language Models
MLPerf	Machine Learning Performance Benchmark Suite
PFS	Parallel File System
GPFS	General Parallel File System
RAGAS	Retrieval Augmented Generation Assessment
ROUGE-L	Recall-Oriented Understudy for Gisting Evaluation - Longest Common Subsequence
BERTScore	Bidirectional Encoder Representations from Transformers Score
AWS	Amazon Web Services

I/O	Input/Output
vLLM	Virtual Large Language Model
FP32	Single-Precision Floating-Point (32-bit)
FP16	Half-Precision Floating-Point (16-bit)
INT4	4-bit Integer Precision
INT8	8-bit Integer Precision
GPTQ	Generalized Post-Training Quantization
AWQ	Activation-aware Weight Quantization
KV-cache	Key-Value Cache
KILT	Knowledge Intensive Language Tasks

Contents

List of Figures	5
List of Tables	6
1 Introduction	7
1.1 Introduction	7
1.2 Motivation	8
1.3 Problem Definition	9
1.4 Research Objectives and Scope	10
1.5 Research Questions and Hypotheses	12
1.6 Research Methodology Overview	13
1.7 Expected Contributions	15
1.8 Structure of the Thesis	16
1.9 Summary	16
2 Background and Theoretical Foundations	18
2.1 Introduction	18
2.2 Large Language Models (LLMs)	18
2.2.1 Overview	18
2.2.2 Transformer Architecture	20
2.2.3 Development Paradigms and Their Effect on LLM Performance	22
2.3 Retrieval-Augmented Generation (RAG)	23
2.3.1 Concept and Motivation	23
2.3.2 Architecture and Workflow	24
2.3.3 Retrieval Techniques	25
2.3.4 Advanced Techniques Explored in the Implementation	27
2.4 High-Performance Computing (HPC) Systems	28
2.4.1 Overview	28
2.4.2 Architecture and Components of HPC	28
2.4.3 Parallelism, Caching, and Batching in RAG Workloads	29

2.4.4	Job Scheduling and Resource Management	29
2.4.5	Containerization	30
2.4.6	Performance Optimization	30
2.5	Evaluation Metrics in Model Deployment	31
2.5.1	System Performance Metrics	32
2.5.2	Retrieval Quality Metrics	33
2.5.3	Answer Quality Metrics	33
2.5.4	Local versus HPC Comparison Metrics	34
2.6	Summary	35
3	Literature Review	36
3.1	Introduction	36
3.2	LLM Deployment Approaches	37
3.2.1	Local and Cloud-Based Deployments	37
3.2.2	HPC-Based Deployments	38
3.2.3	Model Serving Frameworks	38
3.3	Optimization Techniques for LLM Inference	39
3.3.1	Quantization and Pruning	39
3.3.2	Knowledge Distillation	40
3.3.3	Parallelization and Sharding	40
3.3.4	Caching and Memory Optimization	41
3.4	RAG Systems and Optimization Strategies	41
3.4.1	Existing Frameworks	41
3.4.2	Retrieval Efficiency and Hybrid Approaches	42
3.4.3	RAG on HPC Environments	42
3.5	Benchmarking and Evaluation Studies	42
3.6	Identified Research Gaps	43
3.7	Summary	44
4	Methodology	45
4.1	Introduction	45
4.2	Research Framework and Design	45
4.3	Dataset and Input Documents	46
4.4	System Architecture	47
4.5	Implemented RAG Pipelines	48

4.5.1	Baseline RAG version	49
4.5.2	Version Using a Vector Database RAG	49
4.5.3	Tuned Version Using a Vector Database RAG	49
4.5.4	Reranking RAG	49
4.5.5	Hybrid Retrieval RAG	50
4.5.6	Hybrid Retrieval With Reranking RAG	50
4.5.7	Measured Variables	50
4.6	Evaluation Framework	51
4.6.1	System Performance Metrics	51
4.6.2	Retrieval Quality Metrics	52
4.6.3	Answer Quality Metrics	52
4.6.4	Visual Analysis	52
4.6.5	Cross-Environment Comparison	53
4.7	Reproducibility and Documentation	53
4.8	Summary	53
5	Experiments	54
5.1	Introduction	54
5.2	Experimental Setup	54
5.2.1	Software Environment	54
5.2.2	Models and Services	55
5.2.3	Input Corpus	55
5.2.4	Configuration and Parameters	56
5.3	Deployment Stages	57
5.3.1	Local Execution Stage	57
5.3.2	HPC Execution Stage	57
5.4	Experiment Scenarios	57
5.4.1	Scenario-Based Query Execution	57
5.4.2	Baseline RAG Experiment	59
5.4.3	Vector Database RAG Experiment	59
5.4.4	Tuned - Vector Database RAG Experiment	59
5.4.5	Reranking Experiment	60
5.4.6	Hybrid Retrieval Experiment	60
5.4.7	Hybrid Retrieval with Reranking Experiment	60
5.5	Benchmarking and Data Collection	60

5.6	Data Validation	61
5.7	Summary	61
6	Results and Discussion	62
6.1	Introduction	62
6.2	Results of the Local Runs	62
6.2.1	Local Timing Comparison	63
6.2.2	Local Retrieval and Answer Quality	64
6.3	HPC Deployment Results	67
6.4	Local versus HPC Comparison	67
6.5	Effectiveness of Retrieval-Level Optimization Techniques . .	67
6.6	Comparison with Literature	67
6.7	Trade-Offs and Limitations	68
6.8	Summary	69
7	Conclusion and Future Work	70
7.1	Introduction	70
7.2	Summary of Findings	70
7.3	Key Contributions	70
7.4	Limitations	70
7.5	Future Work	70
7.6	Final Remarks	70
	Appendices	76
	Appendix A: Reproducibility and Execution Guide	76
	Appendix B: Evaluation Scenarios and Scoring	77
	Manuals	78
	Installation Manual	78
	User Manual	80

List of Figures

1.1	Overall thesis workflow showing the local deployment stage, the later HPC deployment stage, and the final comparison between RAG variants and execution environments.	14
2.1	Transformer encoder–decoder architecture	20
2.2	Multi-head attention mechanism, illustrating the linear projections of queries, keys, and values, followed by parallel attention heads and concatenation. Adapted from Alammari [1].	21
4.1	High-level RAG workflow used by the implemented notebooks: user query, retrieval over indexed documents, context construction, and answer generation.	47
4.2	Comparison of the five RAG pipeline families implemented in the project: basic RAG, vector database RAG, reranking RAG, hybrid RAG, and hybrid RAG with reranking.	48
6.1	Average local timing breakdown for indexing, retrieval, and generation across the implemented RAG variants.	63
6.2	Answer-quality metrics for the local RAG variants, including manual score, ROUGE-L, BERTScore, and semantic similarity.	64
6.3	Retrieval-quality metrics for the local RAG variants, comparing hit rate, precision, recall, and F1.	65
6.4	Manual answer score by implementation and scenario. Darker cells indicate stronger manually evaluated answers.	66
6.5	Multi-metric profile of the local RAG pipelines, comparing manual score, BERTScore, semantic similarity, retrieval recall, and retrieval precision.	66

List of Tables

2.1	LLM development paradigms and their relevance to this thesis.	23
2.2	Metric groups used in the thesis evaluation framework. . . .	35
6.1	Summary of local evaluation results averaged across the five scenarios.	62

Chapter 1 – Introduction

1.1 Introduction

LLMs were developed as a result of advancements in deep learning that allow them to read, understand and create large amounts of natural language. The development of these LLMs has been directly linked to an architectural advancement known as the Transformer. This architecture allows for the ability to model sequences based upon their relevance (attention) and it allowed for efficient training on larger than ever before data sets, in parallel [2]. With increases in both the amount of data available, computing power and model size, LLMs have achieved impressive results across a variety of tasks including question and answer, generating summaries, coding, and extracting information [3,4]. These capabilities have provided several opportunities for utilizing them for various types of applications; however, with this comes significant technical barriers for deploying LLMs due to the issues associated with memory usage during processing, processing speed/latency for real-time operations, reliance upon other software packages, and ensuring reproducible results.

One technique used to improve both the utility and credibility of LLM systems is called RAGs. The RAG system takes advantage of a retrieval mechanism which pulls in relevant data from an outside body of documents, and then incorporates this retrieved data into the final input (i.e., model prompt) prior to generating an answer [5,6]. Therefore, as opposed to answers being generated using only what is contained within the model’s parameterized knowledge base, the answers are now supported by some form of evidence pulled directly from the outside world. In addition to supporting domain adaptation, RAG can be done with no additional model training or full weight-fine tuning costs [7].

At the same time as producing answers, RAG systems aren’t simply single-model systems. Document loading, chunking, embedding creation, indexing, retrieval (with optional reranking), prompt creation, answer creation, and log creation (i.e. logging results) all occur during the operation of a typical RAG system. Since each of these components can influence both the quality of the final answer created by the system, and the amount of time it takes to

execute the entire RAG system; therefore, in addition to determining if an answer was generated, a successful RAG project will also need to determine how different retrieval strategies impact latency, evidence quality, answer accuracy, and reproducibility.

High-Performance Computing (HPC) provides an interesting backdrop to this research. Because HPC systems often consist of shared compute resources, job schedulers, containers for running code, large amounts of storage capable of supporting large-scale computation, and scalable hardware for running computational workloads [8–10]. Although this thesis limits its experimentation to the retrieval level RAG techniques and how they compare to one another in terms of effectiveness and efficiency; the use of HPC systems in providing frameworks for deployment requirements and scaling in terms of hardware and software to support larger scale workloads, frames our view of practical implementation as extending notebook-based reproducible pipeline workflows toward HPC environments.

1.2 Motivation

The primary purpose behind creating this project is to address the increasing demand for language model applications which use private, domain-specific data or constantly evolving data. A single, self-contained language model will be able to generate coherent responses; however, the language model will likely lack immediate access to the specific documentation required to perform the desired function. Therefore, the model could potentially create responses which seem valid yet would be unsupported by existing documented proof. This is where RAG can help alleviate some of these limitations. By allowing the RAG system to search a previously established body of knowledge and retrieve relevant pieces of text, then pass these retrieved texts into the language model as contextual input prior to generating an answer.

Although the idea of RAG seems relatively simple, the performance and speed of a full RAG solution heavily depends upon how each piece of the solution is implemented. For example, building a very basic in memory retriever is easily accomplished and works well when working with smaller bodies of text. However, using a basic in memory retriever will not provide you with the same level of persistence or deployment options as a Vector Database. For instance, while a Vector Database like Milvus provides much

better persistence options and a much more viable platform for working with large-scale retrieval requirements than an in-memory based approach; they come at an added expense through both set up costs and indexing costs. While reranking will allow you to reorder your retrieved items; reranking will add another model to your overall process and increase your overall retrieval time. Similarly, Hybrid Retrieval allows you to combine semantic and keyword-based retrieval methods; however, it will also add increased complexity to your overall retrieval method.

This leads us to why we created our experimental comparison-in-memory retrieval, vector-database retrieval, vector-database retrieval with optimized chunking, dense retrieval with reranking, hybrid retrieval and finally hybrid retrieval with reranking. We were also motivated by the need for a reproducible workflow due to many service providers, models, notebooks, configuration files, results logs, evaluation scripts and generated images make up a practical RAG solutions.

1.3 Problem Definition

The general problem addressed by this Thesis is how to easily and practically test different versions of a RAG pipeline under a deployment workflow that is fully reproducible. The primary issue is not whether RAG will improve general language-model applications as other studies have shown [5,6], but instead, what effect do different retrieval and ranking strategies have on speed, the amount and quality of retrieved evidence, and the final quality of answers and how easy it is to reproduce these results.

A RAG pipeline contains several components whose behaviour must be coordinated. Documents must be loaded into memory and splitted into chunks; embeddings must be generated; indexes or document stores must be created; candidate passages must be retrieved and finally, using the selected context, the language model must generate an answer. Weaknesses at any stage can reduce the usefulness of the final system. For example, while a fast retriever may return documents that are irrelevant to the user's query, these poorly choosen documents may lead to poor answers. Simultaneously, a re-ranking stage which may enhance the quality of the document choice may also result in significantly longer response times.

Therefore, the primary goal of this thesis is to study the implemented RAG pipelines both as software engineering artifacts and experimental conditions.

The goal of this study is to compare their output using exactly the same input documents, scenario queries, models and logging approach. This creates a controlled framework from which to evaluate the trade-off between performance and output quality.

The wider deployment environment of HPC has constraints such as job scheduling, container management, storage performance requirements and reproducibility requirements. In this thesis these constraints guide the design and documentation of the system. Large-scale multi-node benchmarking will be treated as future extensions.

1.4 Research Objectives and Scope

The main objective of this thesis is to develop RAG pipeline prototypes and execute them in both a local setting and an HPC environment in a reproducible manner. Since we do not aim to find the optimal settings for the model's weights, our focus will remain on the two components of the prototype that can be measured: retrieval methods at the document level and the ways of deploying the prototype as systems.

We attempt to use the same collections of documents, questions, models for languages and embeddings, as well as the same format for logging as much as possible throughout the dissertation. This enables us to measure the influence of different retrieval methods and deployment strategies independently. We have established a base line of behavior using the developed notebooks during the local phase. During the HPC phase, the same files and scenarios were run so that we could compare results from local and HPC environments within a common evaluation framework.

To accomplish the goal outlined in this thesis, it will achieve the five following objectives:

1. **Provide the theoretical and practical foundations for LLMs and RAG Systems.**

The thesis covers the technical foundation necessary for completing the task at hand which includes fine tuning, training from scratch, quantizing, pruning, adapting, etc., so that each of these background techniques can be differentiated from the various retrieval level techniques explored in this dissertation. Additionally, this section includes prompt engineering, RAG, dense and sparse retrieval, hybrid retrieval,

reranking, vector databases, and HPC deployment.

2. Implement and assess the effectiveness of various RAG Retrieval Methods.

The dissertation describes multiple methods for implementing retrieval: a baseline in-memory RAG implementation; a Milvus-based dense-retrieval method utilizing a vector database; a modified version of the previously described method to optimize its performance; a method to use reranked answers with dense retrieval; a hybrid method to combine sparse and dense retrievals; and finally a hybrid method that combines sparse/dense retrievals and uses reranked answers. Each of the implementations is evaluated using standard retrieval metrics, such as hit rate, precision, recall, and F1, along with quality metrics for answers, such as manual correctness scores, ROUGE-L scores, BERTScores, and semantic similarity.

3. Determine the Performance Cost of the Implemented RAG Pipelines.

This section provides the system-wide performance metrics for the RAG pipelines that were implemented in the dissertation through notebook metrics and a log file: indexing time, retrieval time, and generation time. Using the above mentioned metrics will allow researchers to compare the costs associated with retrieving information in memory versus storing information in a database or disk. It also allows for a comparison of the benefits associated with reranking answers, combining sparse and dense retrievals, and optimized chunking.

4. Develop a Workflow to Compare Local vs. HPC Execution.

Prior to executing the HPC portion of the dissertation research effort, local results are collected first. Once local results have been collected, notebooks for local execution are prepared for eventual execution on an HPC platform. The ultimate goal for the HPC phase is to execute both local and HPC versions of the dissertation's evaluation framework, allowing researchers to account for additional overhead factors such as scheduling delays, container startup times, storage access latency, and data transfer costs.

5. Create a Reproducible Evaluation Framework.

In addition to documenting the software tools utilized during execution, the dissertation documents all execution procedures, sample

question sets used for testing, all fields used when logging results, manual evaluation criteria and processes used to manually evaluate results, scripts developed to ensure reproducibility of evaluations, and visualizations created during analysis.

This research limits its study to inference time LLM and RAG loads. This will enable an experiment to focus on retrieving quality answers as well as system performance. The scope does NOT include full model training or large scale distributed training, model fine tuning, model compression (quantization) or pruning, model distillation, and adapter based model optimization. These concepts have been referenced where necessary for a comparison in concept.

1.5 Research Questions and Hypotheses

The research questions are designed to remain within the experimental scope of the thesis. They focus on two measurable areas: the accuracy and answer quality of retrieval-level techniques, and the performance impact of system-level strategies. The local experiments provide the first set of measurements, while the later HPC experiments repeat the same workflow to evaluate portability and deployment behaviour.

- **RQ1: How do different retrieval techniques affect the retrieval-accuracy and answer-quality of the implemented RAG pipelines?**
H1: Hybrid-retrieval will reach a better retrieval-coverage than dense-retrieval due to lexical-matching combined with semantic-similarity; Reranking will make sure that retrieved passages are ordered well and thus increase top-k precision, answer-groundedness, and answer-correctness. It is assumed that the combination of hybrid-retrieval and reranking will lead to the best retrieval-quality-results particularly for those queries which need an understanding of their semantic interpretation as well as the correct wording. Another expected noticeable improvement is the tuned Vector database implementation.
- **RQ2: What kinds of trade-off regarding the performance are introduced through hybrid-retrieval and reranking?** **H2:** The advantages at the level of retrieval are not cost-free in terms of execution-time. As a result, it is assumed that the retrieval-latency of the two basic (vector-database) pipelines will be lower than the one introduced by reranking and by hybrid-retrieval-plus-reranking. Consequently,

the best-performing pipeline concerning answer-quality might not be the fastest pipeline concerning end-to-end latency.

- **RQ3: How do system-level strategies like caching, batching, and parallelism influence the performance of RAG pipelines?** **H3:** Caching is assumed to lower repeated query retrieval-time as well as repeated index loading-overhead without changing the retrieval accuracy. Batching is assumed to improve the throughput if several queries are processed simultaneously. However, there is also a risk that batching leads to increased latencies for individual queries under low workload conditions. Parallelism is expected to reduce the indexing time and the wallclock time of batch evaluations more clearly than it reduces the single query generation latency since the generation still depends on the runtimes of the language model.
- **RQ4: How behaves the same RAG workflow when moved from local-deployment to HPC deployment?** **H4:** If the same files, scenarios, models, and log-format are employed, then retrieval accuracy and answer quality are expected to stay relatively similar in local and HPC execution. Differences are expected to show up mainly in performance and operationally. On HPC, it is expected to improve throughput and batch-execution when parallelism and batching are employed. However, single-query latency might not always benefit since scheduling-delays, container-startup-times, shared-storage-access times, and data-transfer overheads could compensate for the additional computing-resources available.

These hypotheses are deliberately comparative and quantifiable. Every one of them can be assessed utilizing the established evaluation framework: retrieval-quality metrics, answer-quality rubrics, timing fields, throughput measurements, resource measurements, and environment labels for local and HPC execution.

1.6 Research Methodology Overview

The methodology includes a literature study, a software implementation, controlled experiments, and an analysis of the qualitative output from these studies.

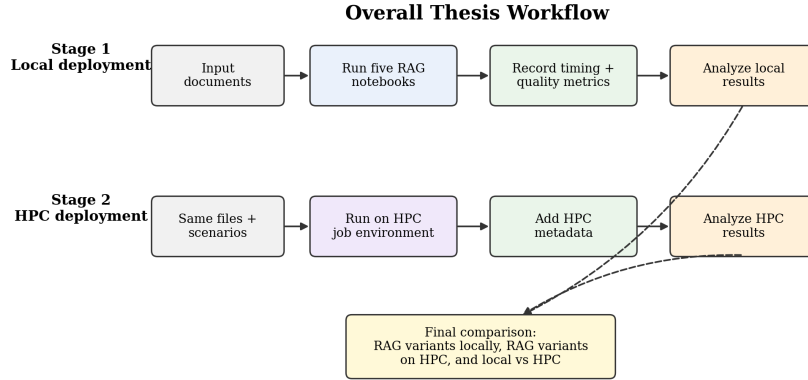


Figure 1.1: Overall thesis workflow showing the local deployment stage, the later HPC deployment stage, and the final comparison between RAG variants and execution environments.

The 1st Step develops the theoretical and technological basis for the studies. It reviews LLMs, transformer architectures, how prompts are generated, RAG architectures, retrieval technologies, reranking, hybrid retrieval and HPC as a technology for deploying systems. Step one provides the vocabulary and context necessary to understand both the implementation (Step 2) and the results (Step 2 and 3).

Step 2 represents the implementation of the research pipeline. The task-related aspect of this project was organized into a series of six Jupyter notebooks that implement a basic RAG pipeline, a vector database pipeline, a fine-tuned vector database pipeline, a reranking pipeline, a hybrid retrieval pipeline and finally, a hybrid retrieval with reranking pipeline. Following the same general format as one another, the content of each notebook includes; loading documents, preparing chunks, creating or connecting to the appropriate store (vector database), running query scenarios, generating answers and saving results.

The 3rd step represents the evaluation process for each of the pipelines. To evaluate each pipeline, timing information is collected. The timing information collected from each of the pipelines is based on seconds. Indexing time, retrieval time and generation time are all included in this measurement. Additionally to collecting qualitative measurements of how well each pipeline performed, the source documents retrieved and the generated answers were also manually scored and analyzed utilizing deterministic metrics. The manual scores represent if the generated answer is correct or incorrect as determined by the evidence within the document corpus. Deterministic metrics utilized include retrieval hit, retrieval precision, retrieval recall, re-

trieval F1 score, ROUGE-L, BERTScore, semantic similarity and length of answer. Collecting both qualitative and quantitative measurements allows for an assessment of the trade-offs referenced in Chapter 6.

The three primary research objectives developed into this thesis were:

1.7 Expected Contributions

1. **Development of a local, reproducible framework for evaluating RAG pipelines, which will serve as a basis for later high-performance computing (HPC) evaluations** The authors provide a defined process for running different RAG implementations using the same set of input documents, question scenarios, log parameters and measurement methods. While all of the empirical data used in the current study has been gathered at the local level; the approach described here could be applied to the HPC platform in future studies.
2. **Empirical comparison of RAG techniques at the document level** In addition to comparing basic in-memory search (retrieval), Milvus-based vector database search, optimized vector database search, dense search with reranking, hybrid sparse-dense search, and hybrid search with reranking; we compare them based on the metrics that have been built into our project: number of hits found by the retrieval mechanism, number of relevant items returned by the retrieval algorithm, number of correct answers provided by the retrieval algorithm, retrieval F1, manually scored answers, ROUGE-L scores, BERTScore values, similarity values between semantically equivalent concepts, and lengths of retrieved answers.
3. **Practical assessment of the trade-offs between quality and efficiency in executing RAG locally.** We measure how the choice of retrieval impacts the times required to index documents, retrieve information from those indexes, generate responses based on the retrieved information and produce retrievable evidence.

Future directions include investigating larger-scale system-wide alternatives (e.g., caching mechanisms, batch processing capabilities, ability to execute multiple processes concurrently, measuring throughput of individual components, memory usage monitoring) as well as comparing execution at both the local and HPC levels.

1.8 Structure of the Thesis

The rest of the thesis will be structured as follows:

Chapter 2 will discuss the theory and history of the background. The chapter covers: LLMs, the architecture of Transformer, training and adaptation strategies for LLMs, prompt engineering, the RAG architecture, retrieval methods, and evaluation metrics for HPC.

Chapter 3 will review other works about: deployment of LLMs (including their optimizations during inference), systems based on RAG architectures, techniques to retrieve information, and benchmarking methods. Chapter 3 identifies what motivated the implementation of the strategy for the evaluation presented in this thesis.

Chapter 4 will present a description of methodology. This includes the research framework, input data, system architecture, implemented pipelines based on RAG architectures, experimental parameters, performance measures, answer-quality measures, and reproducibility procedures.

Chapter 5 provides an overview of how experiments were set up and executed. The chapter provides details on the software platform, models and services, input resources, the order in which notebooks are executed, experiment scenarios, and logging procedure for results.

Chapter 6 presents and examines results. The chapter compares the various pipelines implemented with regard to time-based metrics and output-quality metrics; it also analyzes the trade-off between speed, retrieval quality, answer quality, and reproducibility.

Chapter 7 will summarize the conclusions from this thesis. The chapter will outline the main contributions of this thesis; it will provide examples of its limitations; and it will propose potential future directions for this area of study, especially large scale deployments of HPC systems, additional datasets, and more advanced evaluation methodologies.

1.9 Summary

This Chapter discussed the reason for this study, the problem, the objective of this study, the Research Questions, Methodology, Potential Contributions, and Structure of the Thesis. In order to further narrow down the scope of this study to be only on the RAG pipelines that were implemented, as well

as their trade-off's in terms of what is retrievable at each level (retrieval), the quality of outputs produced by these pipelines, and Reproducibility. The next Chapter will cover all of the Theory behind Language Models, Retrieval models and HPC systems that are necessary for a thorough understanding of the concepts used within this project.

Chapter 2 – Background and Theoretical Foundations

2.1 Introduction

This chapter provides an overview of what you need to know to develop and analyze RAG pipelines used in this dissertation. It does NOT address how to train or tune the parameterization of a language model. Rather, it focuses on the aspects most directly related to the inference time of an RAG system: the Transformer architecture for contemporary LLM; retrieval; reranking; hybrid search; strategies for executing at a system level; and evaluation metrics.

This chapter begins with a description of LLMs. It will explain the Transformer architecture in practical terms. Additionally, although training, fine tuning, prompt engineering, and training from scratch are addressed in the chapter as paradigms for developing LLMs they will be described such that we can distinguish them from the retrieval level and system level methods which are being developed in this work. Next, the chapter will describe RAG, the process flow of RAG, retrieval approaches and other sophisticated retrieval techniques examined in the project source code. Last, the chapter will provide an overview of HPC hardware configurations and discuss the common metrics applied to measure performance, retrieval accuracy, and quality of output.

2.2 Large Language Models (LLMs)

2.2.1 Overview

LLMs are deep learning-based language models which enable processing and generating natural language. Generally these models are trained on massive datasets of text data and statistically learn relationships in natural language, including word, phrase, concept, and context relationships. Due to the nature of their training, LLMs can be utilized for various ap-

plications, including question-answering (QA), document summarization, text generation, and extracting information from documents without explicit task-oriented programming.

While the effectiveness of using LLMs for real-world applications is directly influenced by the model itself; however, there are numerous factors that influence the effectiveness of the model within a larger system architecture. When utilizing LLMs, each time an inference operation is requested, the user provides an input query or prompt. After receiving the input prompt, the model processes the prompt layer-by-layer (many times) through a series of transformer layers and generates output tokens one at a time. While not all operations require significant computational resources, operations involving longer prompts or multiple simultaneous requests may consume considerable resources. Consequently, there are several system level considerations that can have a substantial impact on the performance experienced by users of LLM-based systems. These include batching input requests together, caching intermediate results, selecting appropriate hardware and designing the retrieval mechanism.

This research focuses on utilizing an LLM as part of a RAG system. Therefore, while the primary experimental focus will not involve improving the weights within the LLMs; rather, it involves examining how different approaches to retrieving additional contextual information and overall system configurations effect both the quality and efficiency of the final generated answer.

2.2.2 Transformer Architecture

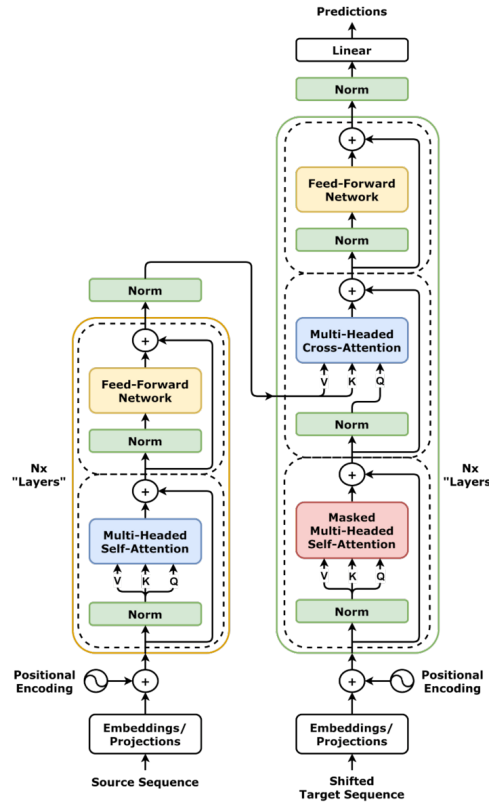


Figure 2.1: Transformer encoder-decoder architecture. The diagram shows the main building blocks used to transform input tokens into contextual representations and generated output tokens.

Modern LLMs are built upon the Transformer Architecture introduced by Vaswani *et al.* [2]. The transformer architecture may be viewed as a sequential processing model, where a sequence of tokens is read into the model, and the representation of each token is iteratively updated based on the representations of the surrounding tokens. This is significantly different from earlier recurrent models that were primarily sequential in their operation, and therefore lacked much inherent or explicit parallelism.

The most important idea in the Transformer is *self-attention*. In simple terms, self-attention is the mechanism that helps the model decide which words in the input are important for understanding each other. For example, in the sentence “the server restarted because it ran out of memory”, the word “it” should be linked more strongly to “server” than to unrelated words. Self-attention allows the model to make these links automatically. Each token is compared with the other tokens in the sequence, and the model gives higher weight to tokens that provide useful context.

More technically, the model creates three learned representations for each token: a query (Q), a key (K), and a value (V). The query represents what a token is looking for, the key represents what each token offers, and the value contains the information that will be passed forward if that token is considered relevant. The attention score is calculated as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \quad (2.1)$$

where d_k is the dimensionality of the key vectors. The softmax operation converts similarity scores into weights, and these weights determine how much information from each token is included in the updated representation.

The Transformer extends self-attention through *multi-head attention*. Instead of applying attention only once, the model uses several attention heads in parallel. One head may focus on grammatical structure, another may focus on entity relationships, and another may focus on long-distance dependencies. The outputs of the heads are then combined. This gives the model a richer representation of the input than a single attention calculation would provide.

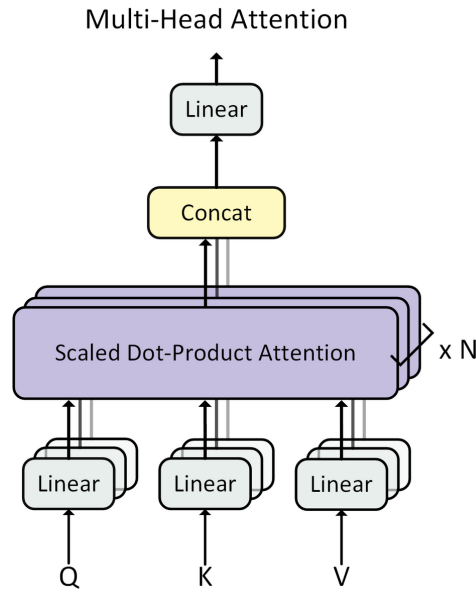


Figure 2.2: Multi-head attention mechanism, illustrating the linear projections of queries, keys, and values, followed by parallel attention heads and concatenation. Adapted from Alammur [1].

The most important aspect of how the Transformer affects the way it behaves during inference, is through its influence on generation. Typically, when using a model for generating something (such as an answer), the generation process is done in a sequential manner. That means the model will produce

one item at a time. Therefore, long prompts, large amounts of data that have been retrieved, and/or multiple questions asked could potentially slow down the overall generation process. So, the RAG evaluation should consider both if the generated answer was accurate as well as the time taken to generate the answer, the time taken to retrieve the relevant information used by the model to generate the answer, and the total amount of time from start to finish (end-to-end).

2.2.3 Development Paradigms and Their Effect on LLM Performance

There are several ways you can create applications based upon LLMs. Each application development method has the potential to significantly impact how your model acts and performs within your system. For this research, these methods were discussed in the background section to clearly define what this research was going to investigate. The experiments did not include optimizing models in terms of model size; fine tuning, quantizing, or pruning any models trained prior to running the experiments. The focus of the experiment is on how to best use retrieval strategies and optimize systems in general.

Table 2.1: LLM development paradigms and their relevance to this thesis.

Paradigm	How it affects LLM performance	Role in this thesis
Prompt engineering	Changes the instructions and context given to a fixed model. It can improve answer style and task following, but it does not update the model’s knowledge.	Background only; prompts are kept controlled across pipeline comparisons.
RAG	Adds external retrieved context to the prompt. It can improve factual grounding and domain specificity without changing model weights.	Main focus; several retrieval variants are implemented and compared.
Fine-tuning	Updates some or all model parameters using task-specific data. It can improve task performance but requires additional training resources and model management.	Out of experimental scope; discussed only to distinguish it from RAG.
Training from scratch	Learns all model parameters from large datasets. It provides maximum control but requires very large compute, data, and engineering resources.	Out of scope; included only as background.

This distinction is important because retrieval-level techniques improve the information supplied to the model, while model-level techniques modify the model itself. The project contract requires understanding the difference between prompting, RAG, fine-tuning, and training from scratch, but the implementation work focuses on deployable RAG pipelines and their performance in local and later HPC environments.

2.3 Retrieval-Augmented Generation (RAG)

2.3.1 Concept and Motivation

RAG uses a combination of a language model and an external retriever. In this approach instead of only getting the answer from the language models own parameterized knowledge base it first identifies appropriate passage(s) from a document collection, which are used as additional input to provide contextual support to the language model. Using the RAG method has been

shown to improve factual grounding and decrease the number of unsupported answers produced by the model especially when the query is specific to a particular domain or type of data that may be less frequently represented within the models' parameterized knowledge base [5].

In terms of using RAG as part of this dissertation the reason was largely practical. It's generally easier to update a document collection vs fine tuning/retraining a model. Therefore, a RAG style system would allow us to train on our project documents, manual, policy etc. or synthetic data sets by simply updating the index of the documents we're searching through; without having to modify the weights of our models. That being said, that increased flexibility does introduce some new performance related questions. For example, how do you load documents? How do you break down each document into manageable pieces? What format will your embeddings be in? Will you need to run another search to find the most relevant chunk(s), rerank if needed etc. Construct a prompt from the retrieved content and generate an answer.

2.3.2 Architecture and Workflow

A high level RAG workflow has two stages: index stage and query time stage. In index phase: documents are loaded, cleaned (removing special characters etc.), split into smaller pieces called "Chunks", embedded using embedding tools such as transformers or bert, and stored in a retrievable structure called document store or vector database. At query-time, user question is processed by retriever to select relevant Chunks. Context is added back into prompt and LLM generates final answer.

Summary of workflow below:

1. **Preparation of Documents:** Source documents collected, cleaned and split into smaller "Chunks" that can be fitted into model's context window.
2. **Indexing:** Chunks stored in retrieval structure. Dense systems store embeddings, sparse systems store lexical statistics. Hybrid systems use both methods.
3. **Processing of Queries:** query from user converted to representation needed by retriever.

4. **Retrieval:** top candidate Chunks returned based on lexical similarity, embedding similarity or combination of both.
5. **[Optional] Re-ranking:** re-ranker re-ranks initial candidates so most relevant Chunks will appear first.
6. **Construction of Context:** selected Chunks inserted into prompt with original question.
7. **Answer Generation:** LLM generates answer using question and retrieved context.
8. **Evaluation and Logging:** system logs timing, quality of retrieval, output-quality metrics for comparison.

High-level view of pipeline is important because each implemented pipeline only changes some parts of workflow. For example baseline-pipeline uses simpler retrieval method than other pipelines. Vector-database pipeline changes storage mechanism, reranker pipeline adds second ranking step, hybrid pipelines combine sparse and dense retrieval signals.

2.3.3 Retrieval Techniques

When selecting the best possible retrieval quality for RAG, there will always be some limitations to how well the retriever does its job. When the retriever finds very little useful context, even though the generator has the capability to create a correct answer (if the language model itself is adequate), then the generator will likely have no choice but to create a weakly-supported, possibly incomplete answer. The retrieval techniques examined during this research were dense retrieval, vector-database retrieval, reranking, and hybrid sparse-dense retrieval. As a supporting concept, sparse retrieval was included here due to it being a component within the hybrid retrieval design.

1. **Sparse Retrieval:** Employs lexical signals, such as exact terms and term frequencies, to align queries with documents. Best Matching 25 (BM25) is a prevalent algorithm in this domain, effective when queries and documents share key terms. However, its reliance on exact matches can hinder retrieval of sections with similar ideas expressed differently. Although not tested independently in this study, BM25 representations were incorporated into all three hybrid-retrieval variant pipelines [11].

2. **Dense Retrieval:** Dense retrieval uses high-dimensional embeddings for queries and documents, focusing on semantic similarity rather than exact wording. This approach allows for the identification of semantically equivalent concepts, exemplified by the Dense Passage Retrieval (DPR) algorithm. In the study, dense retrieval employs embedding based searches on in-memory or Milvus-backed document repositories [12].
3. **Vector Database Retrieval:** Vector databases enable the storage of embeddings and performing nearest neighbor queries on extensive collections of documents, making them particularly beneficial for large, persistent document repositories. This study employs Milvus for the vector database pipeline, specifically the Milvus-backed version of the dense retrieval variant. This type of vector search is closely related to approximate-nearest-neighbor indexing approaches such as Facebook AI Similarity Search (FAISS) and Hierarchical Navigable Small World (HNSW) [13, 14].
4. **Hybrid Retrieval:** Hybrid retrieval integrates sparse and dense features for enhanced performance. Sparse retrieval specializes in exact terminology, while dense retrieval is adept at recognizing semantic similarities. This combination improves recall and robustness across different query types. The study utilizes Milvus’s hybrid retrieval function for implementation.
5. **Reranking:** Reranking is a subsequent stage of information retrieval where an initial candidate list of documents, produced by a retriever, is scored for relevance. Although cross encoder rerankers offer greater accuracy compared to basic vector similarity, they introduce extra computational demands. This study applies reranking to both dense and hybrid retrieval methods [15, 16].

A major tradeoff associated with developing a system for performing retrieval is that stronger retrieval systems typically require longer times to execute. On the other hand, although hybrid retrieval and reranking may provide increased flexibility in retrieving contextual information by improving context selection relative to simpler retrieval systems, these approaches result in slower execution times. This trade-off represents one of the primary measures of success developed during this research.

2.3.4 Advanced Techniques Explored in the Implementation

Although the advanced approaches covered in this paper are limited to the retrieval-level methods that may be implemented and tested in the project’s code, other model-level optimization techniques (such as quantization, pruning, parameter-efficient fine-tuning, prompt engineering, and training from scratch) are outside of the experimental scope [17–20].

The retrieval-level techniques examined include:

- **Dense Vector Retrieval:** a baseline for semantic retrieval based on embeddings.
- **Vector-Database Retrieval:** dense persistent retrieval using Milvus.
- **Retrieval from a Tuned Vector-Database:** a variant of Milvus-based that alters both chunk-length and overlap.
- **Reranking:** ranks retrieved passages before passing them to the generator.
- **Hybrid search:** combinations of dense semantic retrieval with lexical signal style BM25-style sparse searching.
- **Hybrid search with Reranking:** combination of hybrid retrieval with reranked results.

System level metrics measured by the current implementation include: indexing time, retrieval time, generation time. These metrics allow comparison of retrieval strategies’ costs when executed locally.

As future extensions, other system-level strategies (caching, batching, parallelism) are relevant for an HPC deployment stage where embedding work may be repetitive, batch execution can be used to measure job arrays, and concurrent scenario evaluation could be measured more directly.

These techniques match the implemented notebook variants and the current experimental work. This paper focuses on measuring retrieval behavior, answer quality, and execution costs instead of optimizing internal language model parameters.

2.4 High-Performance Computing (HPC) Systems

2.4.1 Overview

HPC systems are collections of interconnected computers (usually referred to as nodes) which have been designed to support computationally intensive workloads requiring greater computational power, memory, and/or data storage throughput than can be provided by a single desktop computer. For this project, HPC is important since the same RAG workflow used locally could potentially be migrated to a managed cluster environment where it would be tested in a variety of differing resource conditions.

In contrast to a typical desktop workstation, HPC systems do not typically execute the users' requests immediately. Instead, when a user logs into an HPC system they will use a login node to create their own directories, prepare their code and job submission scripts and then submit their job to a scheduler (such as Slurm) [9]. At this point the scheduler determines what resources (compute nodes, Central Processing Units (CPUs), Graphics Processing Units (GPUs), etc.) should be allocated to each submitted job based upon predetermined policies of the system. Although HPCs are extremely useful for conducting reproducible batch experiments; the process of doing so introduces operational considerations (e.g., queue time, startup overhead associated with submitting a job, accessing storage, creating containers).

2.4.2 Architecture and Components of HPC

The HPC system consists of multiple parts.

- **Local Workstation:** This is the users' personal computing device which they will utilize to write their codes, interface with the cluster, transfer files and review results.
- **Login Node (Gateway):** The gateway into the cluster, this is where you would edit your files, manage your environments and submit your jobs. Do NOT run your heavy experimental tasks here.
- **Scheduler/Resource Manager:** Receives job submissions, queues them, assigns resources and executes them on the compute nodes. Examples of schedulers/resource managers include Slurm .

- **Compute Nodes:** The nodes upon which the actual workload will be executed. There are some compute nodes that have no GPU, but there are also many compute nodes with one or more GPUs and/or other accelerators.
- **Shared Storage:** Is utilized by the cluster to store all shared items such as datasets, notebooks, containers, vector indexes, logs and result files. Shared storage is critical to RAG experiment workflow performance due to repeated reads from index and document collection.
- **Software Modules and Containers:** HPC clusters provide mechanisms to reproduce software environments via both module systems and containers. Container execution systems like Apptainer are popular in HPC as they support running securely within an unprivileged context.

2.4.3 Parallelism, Caching, and Batching in RAG Workloads

The concept of parallelism is largely defined by how many different systems execute simultaneously versus how many models train with simultaneous data (i.e., how many models train simultaneously). Examples of key types of parallelism include independent query parallelism, batch evaluations, parallel document processing and parallel embedder generation. An example would be dividing up a series of test questions among different worker nodes/submitting each question as its own job. Likewise, a large document could be broken into smaller pieces (chunks) and embedded in batches; not piece-by-piece.

Similar to parallelism, caching is critical for RAG pipelines which contain much duplicated effort. When you use the same documents across a series of experiments you will likely find that your embeddings and indexes can be reused. As well, if you evaluate the same query set on numerous occasions you can cache repeated results for debugging purposes or for comparing the performance of similar systems. While caching can significantly increase speed, care must be taken to ensure that all cached and noncached results are properly accounted for when making comparisons.

2.4.4 Job Scheduling and Resource Management

HPC Execution is dependent on job scripts that specify how much HPCs resource each experiment requires (for example, how many nodes, CPUs,

GPUs, etc., memory, what is the run-time, where does the output go). Then, the Scheduler puts the job into a Queue, and starts the job when there is an opening for Resources.

Job Scheduling will be important for the proposed HPC phase of this Thesis since it could impact both the Performance, and Reproducibility of the Experiments. If measuring end-to-end wall clock time from outside the Application, it would also capture all of the time spent waiting in line with the Scheduler; whereas, application run time measures only the time that the Application spends running once started. Therefore, this methodology will have to separate out Queue Time, Container Startup Time, Indexing Time, Retrieval Time, Generation Time, and Total Application Runtime.

2.4.5 Containerization

Containerization will also enable reproducibility by packaging software dependencies into a portable environment, which will allow for the local experiment to use python virtual environments, Docker, ollema and Milvus while an HPC experiment may require Apptainer or some Scheduler approved container workflow. The goal is to keep the same project files, models, datasets and logging format across all environments and adapt only the system setup required by the cluster to run on it.

For RAG workloads, reproducibility relies on consistent python packages, model versions, embedding model versions, vector database configuration, query sets and result logging. Containerization will reduce the amount of environment drift between the local and HPC stage

2.4.6 Performance Optimization

The HPC performance optimization for this thesis is defined as a system issue. Most influential on performance include:

- **Index Loading/Storage Access:** Indexes and/or collections of documents may have a high cost associated with them when being loaded into memory from shared storage. This can significantly reduce the need to repeatedly pay this overhead by copying data that is accessed frequently to local storage on each node, or by maintaining indexes that remain "warm".

- **Embedding/Generation Parallelism:** Due to the fact that many operations are independent (document chunking, generating embeddings, etc.), document chunking, embedding generation, and evaluation queries can often be executed in parallel.
- **Batch Size:** Batching larger quantities of documents can improve the throughput for either generating or retrieving embeddings; however, it may also increase the amount of memory required per batch, which can result in longer wait times before a query will be processed.
- **Caching Policy:** By reusing cached versions of embeddings, indexes, and repeated retrieval results, we can avoid unnecessary computations. In all future evaluations, we recommend enabling caching if possible. Furthermore, we suggest recording whether caching was enabled during the evaluation process.
- **Resource Allocation:** Selecting the correct combination of CPU, GPU, Memory and Wall-Time Requests for an Experiment will affect how long an experiment runs, where the Scheduling Algorithm places the experiment within the Cluster, and potentially the overall Performance of the Experiment.

These considerations relate directly to the broad deployment goal of the thesis. The above-mentioned Experiments compared Retrieval-Level methods (Dense Retrieval, Vector Database Retrieval, Hybrid Search, Reranking), while System-Level Strategies (such as Caching, Batching, and executing in parallel) were identified as important Considerations for the planned HPC extension and not as completed empirical comparisons in the existing Codebase.

2.5 Evaluation Metrics in Model Deployment

Evaluating an RAG Pipeline requires more than simply measuring how fast it is. An "incredibly" fast system will be completely useless if it retrieves lots of irrelevant documents or provides no supporting evidence for the generated answers. In order to measure these aspects of the evaluation of an RAG system we are using three categories of metrics: (1) system performance; (2) retrieval quality; (3) answer quality. The first category includes indexing time, retrieval time, and generation time. The second category includes

retrieval hit, precision, recall, and F1. Answer quality is evaluated by combining manual correctness scores with Recall-Oriented Understudy for Gisting Evaluation - Longest Common Subsequence (ROUGE-L), Bidirectional Encoder Representations from Transformers Score (BERTScore), Sentence Embedding Semantic Similarity and answer length [21–23]. The classification as well as the categorization of the used evaluation metrics corresponds to the requirement to measure the behavior of machine learning systems and the quality of their generated language [24, 25]. While this thesis did not create an implementation of the Retrieval Augmented Generation Assessment (RAGAS) Framework, the evaluation of the thesis follows the same general categorization that was developed by the authors of the RAGAS Framework into a separation between retrieval behavior, generated-answer quality, and execution costs [26].

2.5.1 System Performance Metrics

System performance metrics measure how efficiently the pipeline runs. Performance metrics track the efficiency of each processing element (chunker, indexer, LLM) in the overall pipeline process:

- **Indexing time:** The amount of time required to load and index all of your documents; chunk them into smaller pieces; generate an embedding from each piece; and/or create/build/populate your document store.
- **Retrieval time:** The amount of time it takes to find all candidate passages for a query. For the reranked pipelines, the retrieval time will be longer due to the fact that reranking occurs during the retrieval portion of the pipeline within the provided notebooks.
- **Generation time:** The amount of time required by the LLM to produce answers once the prompt has been created.

The above metrics are currently being logged through the three fields `indexing_time_s`, `retrieval_time_s`, and `generation_time_s`. Other broader metrics such as end-to-end latency, throughput, CPU/GPU utilization, memory usage, and storage usage will be relevant to future deployment analysis; however, these other metrics are not separately measured in the current version of the software.

2.5.2 Retrieval Quality Metrics

Retrieval quality metrics measure if the retriever finds the right evidence before it generates an answer. These are especially important for evaluating dense retrieval, vector database retrieval, hybrid search, reranking and hybrid search with reranking.

- **Hit:** Does at least one expected supporting document appear among the retrieved sources?
- **Precision:** What fraction of retrieved documents match manually identified supporting documents?
- **Recall:** What fraction of expected supporting documents appears among the retrieved sources?
- **F1:** Harmonic average of precision and recall.

2.5.3 Answer Quality Metrics

The quality metrics for answers determine how well an LLM's final output will satisfy the requirements to provide a high-quality response. Since there is no requirement that an LLM respond within a certain amount of time, these metrics assess the quality of the LLM's output itself rather than just its execution time.

- **Manually-scored:** An LLM's answer is manually scored (correct=1.0; partially correct = 0.5; incorrect = 0.0) based on the ground-truth answer given for it.
- **Manually-labeled:** A category is assigned to each answer: "correct", "partially correct" or "incorrect".
- **Manually-noted:** Comments provided based on evidence explain why an answer was deemed "missing information"; "contains incorrect statements"; or "lacks supporting reasoning".
- **ROUGE-L F1:** A lexical-overlap-based score (using ROUGE-L) compares the generated answer against the ground-truth answer.
- **BERTScore F1:** Semantic-similarity score calculated from contextualized BERT word embeddings.

- **Semantic-similarity using sentence embeddings:** Cosine-similarity between the generated answer and the ground-truth answer, both represented using sentence embeddings.
- **Length of answer:** Number of words in the generated answer - a supportive descriptive metric.

As such this assessment uses both manual judgments and deterministic metrics. Although the manual notes may include some discussion about groundedness, completeness, relevance, and supported claims that could impact correctness, none of those dimensions are currently being recorded separately as numeric fields in the existing evaluation script.

2.5.4 Local versus HPC Comparison Metrics

When the same files are placed in HPC later, a comparison that separates model/retrieval behavior from environmental behavior is required. If the same data, models and parameters are used for retrieval accuracy and answer quality, they should be generally comparable. However, performance metrics can vary due to how resources are allocated (e.g., scheduling, etc.), how quickly data can be retrieved or stored, as well as scheduler overhead, batching and parallelism in the execution of the code.

At least the following items should be logged when comparing a given pipeline run locally versus on an HPC system: execution environment; hardware description; number of workers executing; batch size executed; cache status; index creation time; retrieval time; generation time; and overall run-time. The goal here would be to allow for a direct comparison of the same pipeline run locally and on an HPC system without having to consider differences in the experimental design.

Table 2.2: Metric groups used in the thesis evaluation framework.

Metric group	Implemented metrics	Purpose
System performance	Indexing time, retrieval time, generation time	Shows how expensive each pipeline is to run in the local setup.
Retrieval quality	Retrieval hit, retrieval precision, retrieval recall, retrieval F1	Shows whether the retriever finds the expected supporting documents.
Answer quality	Manual score, manual label, manual notes, ROUGE-L F1, BERTScore F1, semantic similarity, answer length	Shows whether the final answer is correct and similar to the ground-truth answer.
Deployment comparison	Local execution results; planned HPC metadata such as environment, hardware, scheduler, container, and storage details	Shows how the current workflow can later be extended from local execution to HPC.

2.6 Summary

This Chapter has outlined all the Background to be required for the remainder of this Thesis. It has provided an overview of the Transformer Architecture and Self-Attention in Practical Terms, but kept Training (including Fine-Tuning), Prompt Engineering, Training From Scratch as Background Concepts rather than Experimental Objectives. In addition to describing the RAG Workflow (Retrieval Methods, Reranking, Hybrid Search) and System-Level Considerations that will impact on the Deployment and Reproducibility of a large-scale Information Retrieval System.

Finally, the Chapter has also provided an introduction to HPC systems and their components (Login Nodes, Schedulers, Compute Nodes, Shared Storage and Containers). Also, the evaluation metrics that have been used throughout this project to measure both System Performance, Retrieval Quality and Answer Quality were identified. As part of this project's future work plan, the intention is to carry out comparative studies between local vs. HPC environments.

Chapter 3 – Literature Review

3.1 Introduction

The objective of this literature review is to document and evaluate current knowledge on the use of LLMs, methods for optimizing the efficiency and scalability of inference processes, as well as RAG-based architectures used with various types of computational environments. With LLMs continuing to grow in terms of size and complexity, they are resulting in growing computational requirements that necessitate improved methods for deploying and optimizing them locally or distributively (including HPC) in order to support increased levels of utilization. At the same time, prior work has demonstrated potential benefits associated with using RAG systems to improve the factual accuracy of responses from LLMs, reduce instances of hallucination, etc., thus further study will be necessary to investigate the relative performance characteristics of such systems when utilized under a variety of hardware and/or software configurations.

This Chapter brings together prior literature in three broad categories: (i) deployment approaches for LLMs on Local, Cloud, and HPC platforms; (ii) Optimization Approaches to improve Scalability, Memory Usage Reduction and Throughput Improvement of Inference; and (iii) Retrieval Augmentation Systems which include Retrieval Methods, Indexing Techniques, Context Optimizations. In addition, the Chapter provides a review of Benchmarking Standards and Evaluation Methodologies to Assess Computational and Retrieval Performance of LLMs and RAG Pipelines.

Through compilation of findings from these categories, this Chapter identifies several key gaps and shortcomings in the existing body of literature that will guide the experimental design and methodology of this dissertation. Specifically, it draws attention to the lack of studies examining complete RAG pipelines within an HPC environment, the limited comparative evaluations across different hardware configurations, and the lack of standardization with regard to workflow methodologies for evaluating scalability, retrieval latency, and inference efficiency in HPC architectures.

3.2 LLM Deployment Approaches

3.2.1 Local and Cloud-Based Deployments

Research into the deployment of LLMs have generally been limited to either cloud based infrastructure, and local systems. The main differences between these types of deployments are related to their available resources (such as CPU/GPU), how scalable they can be (scalability), the total cost of ownership, and the performance characteristics of each environment.

Deployments using cloud-based infrastructure utilize the elastic computing resources provided by major cloud vendors such as Amazon Web Services (Amazon Web Services (AWS)), Microsoft Azure, and Google Cloud. These services support the use of distributed inference engines, highly scalable GPUs, and also support the ability to manage model serving applications at scale; thus allowing for reliable execution of very large LLM workloads. Additionally studies have shown that through automated scaling, and distributed storage a high degree of efficiency is possible when executing multi-billion parameter models [27]. Some negative aspects associated with deploying models in the cloud include: the high operating expense (e.g. hourly charges for running the service), Data Transfer Fees (for transferring data across networks), Vendor Lock-In (inability to switch from one cloud provider to another) and Latency (the time it takes for data to travel over the network).

Deployments using local infrastructure require a user to deploy their model(s) to a workstation class GPU or on premises server. Local deployments offer several advantages including: predictable performance; low latency per user; and strong data governance. A significant disadvantage of local deployments is the inability to scale up and down to accommodate multiple users or meet the growing demand for increasingly larger LLM models.

Given the current trade-offs between control (local deployments) and scalability (cloud deployments), there exists no single deployment platform that currently meets the needs of modern large-scale LLM deployments. Therefore, we believe that exploring HPC-based deployments would help to address many of the current disadvantages inherent in both local and cloud-based deployments.

3.2.2 HPC-Based Deployments

High Performance Computing systems offer a potential solution for scaling LLM workload due to their highly connected clusters, high-bandwidth interconnects, and massive amounts of total memory. In contrast to cloud based environments optimized for loosely connected service-based applications, HPC clusters allow for rapid communication between each node, enabling both efficient distributed training and inference.

A great deal of prior work focuses on distributed training frameworks (Megatron-LM [28], DeepSpeed’s ZeRO [29]) that utilize tensor, pipeline, and data parallelism to enable the training of extremely large models; these approaches also inform inference based on distributed models with memory limitations per device.

In addition to the above references, other researchers have examined aspects of deep learning in HPC environments including high-speed interconnects such as InfiniBand and NVLink, the effects of parallel file system Input/Output (I/O) performance on LLM, and the impact of scheduler overhead on performance [30]. However, while many of these studies have identified several of the major factors impacting performance in HPC environments, they typically focused on training rather than inference. Fewer studies have been conducted examining *retrieval*-based workloads which are crucial in RAG systems since both latency of the retrieval process and storage throughput affect overall performance of RAG systems.

Prior research has shown how various types of system level decisions can be used to optimize different ML workloads using containerization (Apptainer), and scheduling (Slurm). However, few studies have extended this type of evaluation to full LLM inference or RAG deployments.

While there is clearly an advantage for scaling LLM workloads utilizing HPC platforms, existing literature identifies significant areas for additional study: limited prior research related to inference performance, minimal prior analysis of vector search and retrieval latency, and no full evaluations of the RAG pipeline within HPC environments.

3.2.3 Model Serving Frameworks

A number of high-throughput model-serving frameworks have been developed to scale LLMs:

The **Virtual Large Language Model (vLLM)** framework was the first to use a *PagedAttention* mechanism to improve Key–Value (KV)-cache efficiency and increase throughput during heavy concurrency as compared to previous mechanisms [31]. The current version of this framework is likely one of the top performing open source inference engines.

The **NVIDIA Triton Inference Server (TIS)** is an inference server framework capable of running multiple frameworks concurrently using dynamic batch sizing and GPU-aware task scheduling. However, it was not initially designed with LLMs in mind, therefore, it has limited ability to optimize LLM-specific parameters.

The **Text Generation Inference (TGI)** framework was created for the purpose of optimizing performance when deploying transformer-based text generation models. TGI supports tensor-parallelized inference, quantization, and continuous batch processing.

RayServe is a distributed orchestration tool for scaling applications across cluster resources. While RayServe does provide a flexible way to deploy applications across multiple machines, its primary function is focused on the distribution of tasks across nodes rather than providing optimizations for inference speed.

Therefore, while there are several serving frameworks that can be used to distribute application logic, few have evaluated the impact of hardware constraints found on HPC systems (e.g., batch scheduling, network fabric, and parallel I/O) on the overall performance of these frameworks.

3.3 Optimization Techniques for LLM Inference

3.3.1 Quantization and Pruning

Quantization allows both a reduction in Memory (Memory Usage) and Computation (Computational Complexity), by converting the Weights and Activations from High-Precision (Single-Precision Floating-Point (32-bit) (FP32) / Half-Precision Floating-Point (16-bit) (FP16)) to Lower Precision Formats like 8-bit Integer Precision (INT8) or 4-bit Integer Precision (INT4). Methods including LLM.int8 [32], Generalized Post-Training Quantization (GPTQ), and Activation-aware Weight Quantization (AWQ) allow for significant reductions in Resource Utilization with minimal losses in Accu-

racy. This enables Larger Batch Sizes, Faster Inference etc.

Pruning is a method to remove redundant Weights from an existing model to reduce the Size of the model and Computational Complexity. Although early research focused primarily on Vision Models, recent work has applied Pruning to Transformer Models, and has demonstrated that Structured Sparsity can be used to reduce Compute Requirements without large losses in Accuracy.

There are both Advantages and Disadvantages for HPC Environments for Quantizing and Pruning, which will Improve GPU Memory Fragmentation, increase Batching capacity, and decrease I/O Overhead on Parallel File Systems (PFSs). There are almost no evaluations of these Techniques under the constraints of HPC.

3.3.2 Knowledge Distillation

Distillation has been shown to allow a larger "teacher" LLM to be reduced into a smaller "student". For example, models like DistilBERT [33] have demonstrated that many capabilities of an LLM may be preserved in a model that is significantly cheaper than the original model.

However, the knowledge distillation method will still result in lower latency and memory usage than the teacher model but it does not provide better results on knowledge-intensive tasks. The method also does not inherently prevent hallucination and does not include the ability to ground retrievals in either the data used or the context in which the model was trained. Lastly, the research has not examined how knowledge distillation methods use HPC resources including allocation of those resources, scheduling, and distribution during inference.

3.3.3 Parallelization and Sharding

Strategies to enable Data Parallelism (DP), Tensor Parallelism (TP) and Pipeline Parallelism (PP), represent the most important ways to scale LLMs. TP and PP as used in Megatron-LM [28] are key to enabling distributed execution. The use of parameter, gradient and optimizer state shards in DeepSpeed ZeRO [29] and PyTorch Fully Sharded Data Parallel (FSDP) can be seen as an extension of the ideas behind those two forms of parallelization.

While there is a substantial body of work analyzing the impact of these methods on the training process, few studies have looked at their impacts when executing inference in a distributed environment, particularly in environments using RAG based pipelines that utilize both retrieval processes and distributed KV-cache implementations along with inter-node communication.

3.3.4 Caching and Memory Optimization

The attention model by FlashAttention [34] reduces the memory consumption while keeping the accuracy to a high level, thus is very suitable for applications with low memory requirements.

The PagedAttention (vLLM) splits the Key-Value Cache (KV-cache) to paged blocks, it improves significantly the performance on the systems which serve multiple requests at once.

In addition to sparsification, eviction strategies and cache compressions, researchers are actively exploring other optimizations. However most of them are still in an experimental phase.

Due to multi-user workloads, job schedulers etc., HPC environments have highly varying amounts of available memory. Despite that, there is hardly any literature about applying this type of optimizations on HPC-systems or during multi-node inference.

3.4 RAG Systems and Optimization Strategies

3.4.1 Existing Frameworks

The major frameworks for building RAG pipelines (LangChain [35], LlamaIndex [36] and Haystack) offer modules to build the pipeline. Yet according to the literature there are very few benchmarks, they have been tested little on a distributed computing environment and most do not support HPC-specific workflow.

3.4.2 Retrieval Efficiency and Hybrid Approaches

Although, there have been advancements in the area of hybrid retrieval techniques that use a combination of both sparse and dense retrieval strategies to enhance recall and robustness, the research community has made little effort to evaluate retrieval performance with respect to the following categories:

- PFSs,
- Multi Node Clusters,
- High Performance Computing Interconnects,
- Retrieval performance using Slurm.

3.4.3 RAG on HPC Environments

While there has been a relatively small amount of research conducted on RAG in High Performance Computing (HPC) environments, most RAG related research is based upon assumptions that are specific to cloud computing and/or enterprise deployments, where services exist persistently, dynamic autoscaling exists, and container orchestration exists.

In contrast, HPC systems utilize batch scheduling (e.g., Slurm) and employ parallel storage systems (e.g. Lustre, General Parallel File System (GPFS)). The jobs executed on these systems do not remain active after their completion. These differences create additional complexities regarding how to serve, retrieve, and perform distributed inference.

3.5 Benchmarking and Evaluation Studies

Benchmarks such as Massive Multitask Language Understanding (MMLU), Holistic Evaluation of Language Models (HELM) [25], and OpenLLM Leaderboard assess LLM quality but ignore system performance. Machine Learning Performance Benchmark Suite (MLPerf) [37] measures inference speed but does not evaluate:

- autoregressive decoding,
- retrieval latency,

- distributed inference.

RAG-specific benchmarks like Knowledge Intensive Language Tasks (KILT) assess retrieval accuracy but not computational behavior or scalability. Across the board, benchmarks do not capture:

- scheduler effects,
- inter-node communication,
- parallel file system performance,
- container overhead.

3.6 Identified Research Gaps

The literature reveals several critical gaps:

- Lack of systematic studies on LLM inference on HPC systems.
- Lack of research on RAG pipelines deployed on HPC platforms.
- Insufficient analysis of Slurm scheduling and multi-user resource contention.
- Limited comparisons of local, cloud, and HPC inference performance.
- No end-to-end evaluation of vector search and retrieval in HPC environments.
- Absence of reproducible workflows for LLM/RAG experiments on HPC.

These gaps prevent practitioners from making informed decisions about infrastructure, optimization choices, and deployment strategies. This thesis addresses these limitations through controlled experiments, cross-environment benchmarking, and analysis of full RAG pipeline performance.

3.7 Summary

This chapter reviewed the literature on LLM deployment strategies, inference optimization techniques, RAG System Design, and current evaluation methods for LLM and RAG. While much has been accomplished in scaling and optimizing LLMs, little research is currently being conducted to study deploying and evaluating LLM and RAG workloads on HPC Systems. Also, there exists an unfulfilled need for benchmarking as well as the analysis of HPC-specific issues (e.g., behavior of interconnects, performance of parallel storage, scheduling) that have never before been studied.

Therefore, these shortcomings will be addressed through the methodology presented within this dissertation. Chapter 4 discusses the experimental setup, deployment strategies, optimization techniques, and metrics of evaluation to measure the performance of LLM and RAG across Local, Cloud, and HPC environments.

Chapter 4 – Methodology

4.1 Introduction

The purpose of this chapter is to explain how the Retrieval-Augmented Generation (RAG) pipelines were designed, implemented, and evaluated for this thesis. This chapter ties together the theoretical background on LLMs, semantic search, reranking, hybrid retrieval and HPC deployment to a very practical notebook based system as it was implemented in the repository.

This project employs an experimental engineering approach to software development.

First a baseline RAG pipeline is created and thereafter retrieval level improvements are added to controlled variants. In order to test whether variations in timing or answer quality are due to the retrieval and ranking strategy under examination, all variants use the same data set, question scenarios, logging procedures, and evaluation programs.

4.2 Research Framework and Design

The approach uses six primary RAG architecture families and one RAG vector-database variant that has been specifically tailored. All six RAG architecture families appear in FIGURE 4.2. The implemented variants include a basic in-memory pipeline, a Milvus vector-database pipeline, a tuned vector-database pipeline, a reranking pipeline, a hybrid retrieval pipeline, and a hybrid retrieval with reranking pipeline. The main focus is to evaluate how these choices affect retrieval quality, answer quality, and execution time. The result logs reflect this as `basic`, `vectordb`, `vectordb_tuned`, `reranker`, `hybrid`, and `hybrid_rerank`.

The comparison is based on a "local first" workflow. The notebooks will be run on local systems to test all aspects of the code, services, prompts, documentation generation, results logging and evaluation scripts. All of these workflows will then be ready to be run in the future on an HPC. While the ultimate goal of running this on an HPC is not to modify the RAG logic of the system, it is expected that some retrieval strategies (such as those

using shared resources, schedulers etc.) may behave differently depending on the specific characteristics of the HPC execution environment (storage locations etc.), compared to their performance when executing within a local environment.

By focusing on retrieving and ranking, and keeping the generator model consistent across experiments wherever possible, we can isolate and compare the effects of changing various components of our retrieval strategy; including dense retrieval, persistent vector storage, tuned chunking, reranking, sparse-dense hybrid retrieval and hybrid retrieval with reranking.

4.3 Dataset and Input Documents

The experiments utilize the OrchestrAI enterprise synthetic dataset that was created for the purpose of this project (stored inside the project directory). The dataset includes 54 .docx files which were organized within departmental directories: (i) Customer Support, (ii) Engineering, (iii) Finance, (iv) HR, (v) IT, (vi) Logistics, (vii) Product, (viii) Sales, (ix) Security, (x) Q&A Handbooks. The contents of these .docx files include company policy descriptions, customer service procedure documentation, on-boarding workflow information, procurement process documentation, product development/production information, cross-function operational scenario documentation.

Each Notebook utilizes the Haystack `DOCXToDocument` converter to load the same document set recursively. While documents are loaded, they are enriched with additional metadata – primarily the original source filename and department. This additional metadata will be useful in manually evaluating retrieved documents when the retrieved documents are saved to the `results.csv` for later comparison purposes.

Once all documents have been loaded into memory, they are divided into separate chunks prior to being indexed. The primary in-memory index pipeline utilizes word-based chunks of 120 words in length with an overlapping chunk of 20 words. The Milvus supported vector database, reranking, hybrid, and hybrid reranking pipelines utilized 80-word chunks with an overlapping chunk of 20 words. A single variation of the Tuned Vector Database Indexing Pipeline changed this parameter to allow for larger chunks (of 200 words) with greater overlap (an overlapping chunk of 40 words), thereby enabling the ability to test/chunk-size as a controlled variable.

4.4 System Architecture

The developed system has followed the common RAG architecture as illustrated in Fig. 4.1. In this case, a query by the user was used to retrieve relevant fragments of documents, these retrieved fragments were then integrated into a prompt which was provided to the language model as input. This prompted the language model to generate an answer based upon that context.

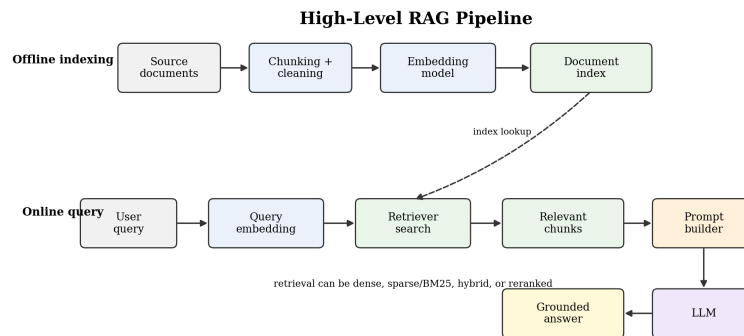


Figure 4.1: High-level RAG workflow used by the implemented notebooks: user query, retrieval over indexed documents, context construction, and answer generation.

Haystack components have been utilized in order to implement all aspects of the system, e.g. document conversion, splitting, embedding, retrieval, prompt construction, etc. Ollama has served as the host platform for both the answer generation model (llama3.1) and the embedding model (mxbai-embed-large). Since the embeddings used for dense retrieval are also created via use of the embedding model, a direct connection exists. The variant utilizing a local Milvus server connects directly to a local instance of the service. These services are managed via use of the standalone script `scripts/standalone_embed.sh` within the project directory. Cross-encoding reranking utilizes a MiniLM cross-encoder trained upon MS MARCO ranking data.

The main architectural components are:

- **Document Loader/Chunker:** Loads .docx files, saves source meta-data from loaded documents and creates "chunks" that represent the document's content broken down by retrieval unit.
- **Embedding Model:** Converts each Document Chunk and User Question into a dense vector representation (e.g., embedding).

- **Document Store:** Stores each chunk in memory or uses a database like Milvus based on how the implementation was set up.
- **Retriever:** Uses Dense Vector Similarity or Hybrid Sparse-Dense Retrieval methods to find the best possible "candidate chunks".
- **Reranker:** Reorders the retrieved Candidate Chunks when using the Reranking variants before passing Context to Prompt.
- **Prompt Builder & Generator:** Combines the Retrieved Context with the User's Question and sends the combined Question-Context pair to Llama3.1.
- **Results logger:** Writes the Implementation Name, Scenario Identifier, Retrieved Sources, Generated Answer and Timing Values to a shared CSV File.

By structuring the code in this way it allows for fair comparison since all Notebooks use the same high-level workflow but differ at the Retrieval/Ranking strategy level. Also, this structure will allow for future HPC execution since all Notebooks and helper scripts may be executed in a managed environment with only environment specific setup changes.

4.5 Implemented RAG Pipelines

Implemented RAG Pipeline Variants

Pipeline	In-memory docs	Vector DB	Dense retrieval	Sparse / BM25	Reranker	Tuned chunks	LLM generation	CSV logging
basic	Y	-	Y	-	-	-	Y	Y
vectordb	-	Y	Y	-	-	-	Y	Y
vectordb_tuned	-	Y	Y	-	-	Y	Y	Y
reranker	-	Y	Y	-	Y	-	Y	Y
hybrid	-	Y	Y	Y	-	-	Y	Y
hybrid_rerank	-	Y	Y	Y	Y	-	Y	Y

Y indicates that the component or configuration is part of the pipeline variant.

Figure 4.2: Comparison of the five RAG pipeline families implemented in the project: basic RAG, vector database RAG, reranking RAG, hybrid RAG, and hybrid RAG with reranking.

4.5.1 Baseline RAG version

The baseline in-memory RAG version, `rag_basic.ipynb`, utilizes an in-memory document store and an embedding based retriever. The goal of this pipeline was to provide the simplest completely working RAG model and serve as a reference point for the other versions. It demonstrated how retrieval and generation work when there is no added overhead of a vector database, sparse retrieval, or reranking.

4.5.2 Version Using a Vector Database RAG

The vector-database version, `rag_vectordb.ipynb`, places document embeddings into Milvus. For each query, the version queries the Milvus collection and returns the top 10 dense matches. This version tested the effects of utilizing persistent vector databases as opposed to in memory document stores.

4.5.3 Tuned Version Using a Vector Database RAG

The tuned vector-database version, `rag_vectordb_tuned.ipynb`, has the same Milvus backed dense retrieval structure as the standard vector-database pipeline. However, the primary differences were related to the chunking settings; specifically, the chunk size went up from 80 to 200 words while the chunk overlap increased from 20 to 40 words. This version evaluated if larger chunks would improve either retrieval behavior or answer quality.

4.5.4 Reranking RAG

The reranking version, `rag_reranker.ipynb`, first retrieves 20 dense candidates via Milvus. Then those candidates are reordered using a cross-encoder reranker. Finally, the top 10 reranked candidates are sent to the generator. This version tested if adding another relevance order step to the initial set of retrieved content would improve its usefulness enough to justify the higher latency of performing two separate retrievals.

4.5.5 Hybrid Retrieval RAG

The hybrid version, `rag_hybrid.ipynb`, used a hybrid retrieval mechanism made available by Milvus. A combination of dense semantic embeddings (i.e., semantic matching) and a sparse BM25 style representation produced inside Milvus (i.e., lexical matching). The goal of this version was to determine if semantic matching and lexical matching could be complementary. This is particularly interesting given that enterprise documents often contain exact term references (e.g., team name(s), severity level(s), product identifier(s), etc.).

4.5.6 Hybrid Retrieval With Reranking RAG

The hybrid-reranking version, `rag_hybrid_&_rerank.ipynb`, built upon the hybrid retrieval version. First, it retrieves 20 hybrid candidates. Next, it uses the cross-encoder reranker to select the top 10 documents for generation. While this version may enhance context ordering due to combining dense retrieval, sparse lexical matching, and reranking; it comes at a cost: the time needed to perform the retrieval will increase since both hybrid candidate retrieval and reranking occur prior to the actual generation of answers.

4.5.7 Measured Variables

We measure the following primary variables.

- Time to index in seconds
- Time to retrieve in seconds
- Time to generate in seconds
- Source files and departments that were retrieved
- Contexts that were retrieved
- Number of retrieved documents
- Top-k retriever and reranker values
- Generated answer text
- Correctness score and label given by manual assessment

- Deterministic measures of answer quality and retrieval quality.

We have logged for future use all of the above (except for correctness assessments) as part of our current Results Logger: Implementation Name, Scenario ID, Question Asked, Answer Given, Sources Retrieved, Departments Retrieved, Contexts Retrieved, Number of Documents Retrieved, Top-k Values For Retriever and Reranker, Indexing Time, Retrieval Time, Generation Time. Manual correctness assessments and labels are included at some point after this has occurred, via the Ground-Truth Evaluation File. The evaluation script also produces deterministic metrics. The end-to-end response cost can be estimated using the total times for both retrieving and generating answers, however, we treat indexing time separately since it represents a cost associated with preparing the corpus before queries are run, whereas the other two represent costs associated with answering each query.

4.6 Evaluation Framework

4.6.1 System Performance Metrics

The overall system performance will be measured by timing that was collected from within the notebooks. The indexing time represents the total amount of time required by the system to prepare and save the chunks from documents. The retrieval time is used to represent how much time it takes to retrieve all the relevant chunks related to a specific scenario-based question. As mentioned previously, the reranking processes are considered to be part of the retrieval time in both cases. Finally, generation time indicates how long it takes for an LLM to generate a final response once the appropriate prompt is generated.

Timing values obtained while running locally will be saved to the experimental results CSV and subsequently to the final results directory.

When comparing HPC systems with RAG systems, we anticipate collecting the same timing fields as before; however, we plan to collect additional timing fields which relate specifically to HPC (i.e., queue time, container start-up, service start-up, data-transfer time) in order to separate them from actual times associated with RAG pipeline execution time.

4.6.2 Retrieval Quality Metrics

Retrieval quality can be measured based on how well the model identifies true supporting documents compared to the manually selected supporting documents provided in the ground-truth CSV. In addition to calculating the proportion of manually selected supporting documents which were actually retrieved (recall), the proportion of all supporting documents retrieved that were also manually selected (precision) and an overall measure of balance between the two (F1). A "retrieval hit" exists when at least one manually selected supporting document is included in the list of retrieved sources.

4.6.3 Answer Quality Metrics

Manual evaluations assess the quality of the answer by providing a score to determine if the response was completely accurate, partially accurate, or completely inaccurate. Manual scoring is typically done by assigning a score of either 1.0 for fully correct, 0.5 for partially correct, and 0.0 for no correctness.

In contrast to evaluating answer quality manually, the deterministic evaluation script calculates several additional metrics that can automatically measure aspects of answer quality. ROUGE-L measures lexical overlap between the reference answer and the model's answer [21]. BERTScore measures the semantic similarity of the model's answer versus the reference answer [22]. Similarly, a sentence transformer can measure the level of semantic similarity between individual sentences within each answer [23]. Additionally, there are metrics related to the retrieval quality of the model such as whether enough supporting documents were retrieved. Lastly, the deterministic evaluation script evaluates the length of each answer.

4.6.4 Visual Analysis

Visualizations are created through the use of a visualization notebook. After reading and processing all of the final results, it produces several types of figures that show answer quality, retrieval quality, per-scenario performance, timing breakdowns, and comparisons across multiple metrics. Each type of figure is then placed into its own subdirectory called "results figures." The figures created in this manner Figures 4.1 and 4.2 are used throughout the results chapter to demonstrate trade-offs made during implementation.

4.6.5 Cross-Environment Comparison

4.7 Reproducibility and Documentation

Although the results logger module standardizes how results are collected by forcing each notebook to collect results in the same format, each notebook can still be run independently. Thus, each notebook represents a unique version of the RAG pipeline being tested. All other modules including the ground truth builder module that builds the evaluation template from generated results, and the deterministic evaluation module that compute all of the automated metrics are also independent.

Documentation is necessary for reproducibility. Therefore, there is a **README** document for setting up the software and running each notebook, along with documenting how to start Milvus and what Ollama models are required, how to execute each notebook, what command to run to perform evaluation, where output files are written etc. Local reproducibility is supported via this **README** document and serves as the foundation for future HPC installation instructions. For HPC reproduction purposes, documentation should include file transfers, environment configurations, scheduler commands, container/-module selection options and/or output CSV file storage location(s).

4.8 Summary

This chapter described the methodology that was employed to test and evaluate the various RAG pipelines developed in this study. The chapter describes the experimental design for testing and evaluating these pipelines, defines how large a dataset was needed for training/testing, defines the corpora, outlines system architecture, lists out variations of each pipeline, describes variables that could affect results, describes how results would be evaluated, describes what types of visual outputs would be produced as part of analyzing results, and finally outlines how reproducible this study was.

In addition to describing methodology, this chapter provides the foundation upon which subsequent chapters build. The next chapter discusses how experiments were performed on these RAG pipelines.

Chapter 5 – Experiments

5.1 Introduction

This chapter details the way in which the experimentation has been conducted. While chapter 4 described both methodology and an evaluation framework, this chapter concerns itself with the actual process of conducting the experiment (software setup, models and services startup, execution of the notebooks, the sequence of scenario questions to be answered, parameterization of all aspects of the experiments, logging results and validating all outputs generated by each experiment).

While the organization of the experiments is based upon the local execution phase, there will also be an equivalent organization for the HPC phase, however, at this time the implementation is focused upon local comparisons and as such the majority of information contained within these result files relates to that aspect. It is necessary to note that it is the local executions which serve as the empirical baseline, whereas the HPC phase should execute the same notebooks and scenarios under identical conditions (i.e., without changing the retrieval logic).

5.2 Experimental Setup

5.2.1 Software Environment

The experiments were implemented as a series of Python Jupyter Notebooks. Each experiment was set up to use a Python Virtual Environment. All dependencies required by the repository were installed. Once the dependencies were installed all the experimentation Notebooks were loaded and run directly from the Project Notebooks Directory. The Local Workflow utilized Ollama for Model Serving and Milvus for both Vector-Database and Hybrid-Retrieval Variants of RAG.

The executed implementation notebooks are:

- `rag_basic.ipynb`;

- `rag_vectordb.ipynb`;
- `rag_vectordb_tuned.ipynb`;
- `rag_reranker.ipynb`;
- `rag_hybrid.ipynb`; and
- `rag_hybrid_&_rerank.ipynb`.

All notebooks followed a similar general process of execution. That process included importing libraries, configuring paths and services, loading the document corpus, splitting/indexing documents, building the RAG pipeline, executing scenario questions, and saving results. This standardized structure allows for the elimination of unwanted variation between implementations.

5.2.2 Models and Services

In order to run locally two Ollama models need to exist. The generation model will be `llama3.1` and the embedding model will be `mxbai-embed-large`. For dense retrieval, vector-database retrieval, and hybrid retrieval, all of these methods rely upon the embeddings. Reranking notebooks also make use of a MiniLM cross-encoder from the sentence-transformers ecosystem.

Prior to running the vector-databases (reranking), hybrid, tuned-vector-databases, hybrid-rerank notebooks, the milvus service is launched locally. A milvus local service is managed by the project script `standalone_embed.sh`. An in-memory baseline can be used without having milvus installed and thus can be used for a reference implementation when you want something simple.

At the HPC stage later in this paper, we wish to maintain as many of the model choices and retrieval configurations as possible. In cases where changes are necessary due to differences in serving methods, container images, file system paths, or scheduler configurations those items should have been noted prior to comparing local versus HPC results.

5.2.3 Input Corpus

All experiments make use of the same OrchestrAI synthetic enterprise corpus. The corpus includes 54 DOCX documents divided by departments. Each notebook will load each document recursively and convert them into a

haystack document format; it also keeps track of source information about each document (file name and department).

Since all notebooks have access to the same set of documents, the primary cause for differences in answers is the way in which they retrieve and rank those documents. The same corpus needs to be duplicated on the HPC system used for the second phase and retain the directory structure and file names.

5.2.4 Configuration and Parameters

The key configurable options include the model types, query types (e.g. "what," "where"), total number of retrieved documents, chunking configurations and logging schema. In most cases we have passed in a total of ten retrieved document-chunks to the language models for evaluation. For the reranked versions we first retrieve twenty candidates using our ranking models, and then filter down to the top ten based on the output of our cross encoder rerankers.

All of the chunking configurations differ from each other only when they were used as part of an experimental design. The baseline used 120 word chunks that had 20 words of overlap. All of the reranking variants including the dense milvus version, the hybrid version and the hybrid-reranking variant all used 80 word chunks with 20 words of overlap. The only chunk size used by the vector database tuning notebook was 200 words with 40 words of overlap. The rest of the vector-database configuration remained unchanged.

- **Top 10 retrieval:** basic, vector database, tuned vector database, and hybrid.
- **Top 20 retrieval followed by top 10 reranking:** reranking and hybrid with reranking.
- **Chunking test:** tuned vector database changes only chunk length and overlap.

5.3 Deployment Stages

5.3.1 Local Execution Stage

The local execution stage represents the final experimental stage of the project. The primary goal of this stage is to validate that each notebook successfully runs on the author’s computer and to provide comparative timing, retrieval and answer quality outputs.

The notebooks were run in the same order as follows:

1. baseline in-memory RAG;
2. Milvus vector-database RAG;
3. tuned Milvus vector-database RAG;
4. dense retrieval with reranking;
5. hybrid retrieval; and
6. hybrid retrieval with reranking.

The above order represented an increasing complexity pipeline, starting with the most basic pipeline and adding persistent vector storage, chunking variations, reranking, hybrid retrieval and finally hybrid retrieval with reranking. All outputs generated during each run are recorded into a single results.CSV file for comparison purposes.

5.3.2 HPC Execution Stage

5.4 Experiment Scenarios

5.4.1 Scenario-Based Query Execution

Five predetermined question-answering scenarios serve as the framework for the experiments. Different retrieval and reasoning behaviours are tested in each case. A straight factual lookup is the first scenario. The latter situations call for cross-document or procedural reasoning.

The five scenario questions are:

1. ***What is the first-response SLA for a Sev-1 support incident?*** This is the clean baseline scenario. It checks whether the system can retrieve and answer a single operational fact accurately.
 - Retrieval type: single-document
 - Main test: exact fact retrieval
2. ***What steps must be completed before a new employee is considered ready for day one at OrchestrAI?*** This scenario is slightly more complex because the answer is not a single fact but a short operational process.
 - Retrieval type: checklist or procedure retrieval
 - Main test: whether the system can recover and summarize a structured process
3. ***A new engineer starts on Monday, but their laptop has not arrived and their required access is still pending. Which teams are involved and what should happen next?*** This scenario requires combining information from more than one team to answer a realistic workplace problem.
 - Retrieval type: multi-document, cross-team retrieval
 - Main test: whether the system can synthesize HR, IT, and access-related responsibilities
4. ***Can OrchestrAI make an urgent hardware purchase without following the normal PO workflow if a warehouse outage is at risk?*** This is the most demanding scenario in the notebook because it requires cross-functional reasoning across several business areas.
 - Retrieval type: policy retrieval with exception logic
 - Main test: whether the system can identify when a standard workflow may be bypassed
5. ***A customer renewal is at risk because support cases are recurring, a requested feature is still unavailable, and replacement hardware is delayed. What should OrchestrAI do next, and which teams must be involved?*** This scenario is slightly more complex because the answer is not a single fact but a short operational process.
 - Retrieval type: multi-hop, multi-document retrieval

- Main test: whether the system can combine support, product, logistics, and commercial risk signals into one grounded answer

The same questions are used for all implementations. Each implementation uses its own scenario identifier prefix so that rows remain easy to distinguish in the results file. For example, the baseline uses b1–b5, the vector-database run uses v1–v5, and the hybrid-reranking run uses hr1–hr5.

5.4.2 Baseline RAG Experiment

In this baseline experiment we execute the five scenarios with our in memory RAG pipeline. We load the DOCX chunks in memory and index them into an in memory document store. For each of the five questions we retrieve the top 10 dense match candidates. This experiment will serve as a reference point for all other experiments presented in this paper. In addition it serves as a baseline for the other experiments.

5.4.3 Vector Database RAG Experiment

This experiment executes the same five scenarios, but now on a persistent vector database (Milvus). Again, we search for the top 10 dense match candidates for each of the five questions. This experiment examines the impact of persistently storing vectors in a database and if the quality of the retrieved matches is still similar to those produced by the baseline experiment.

5.4.4 Tuned - Vector Database RAG Experiment

Additionally, this experiment also includes the execution of the same five scenarios with a larger chunk size and overlap. As previously mentioned, chunk size has a strong influence on the behavior of the retriever and the answers generated. To isolate the effects of chunk size and maintain consistent conditions for the retriever, model, prompt, and top- k value we use the same setting for the retriever, model, prompt, and k value. Therefore we call this experiment "tuned".

5.4.5 Reranking Experiment

For this reranking experiment, we again execute the five scenarios based on dense retrieval (Milvus) but now followed by cross encoder reranking. First, the retriever delivers 20 candidates and then the reranker selects the best 10 candidates for prompting. This experiment assesses the trade-off between additional latency during retrieval stage versus improved ranking order.

5.4.6 Hybrid Retrieval Experiment

The hybrid retrieval experiment combines dense semantic retrieval with a sparse BM25-style signal generated inside Milvus. It retrieves 10 documents for each question. This experiment evaluates whether combining semantic and lexical signals improves robustness for questions that contain exact operational terms such as severity labels, team names, procurement terms, and workflow names.

5.4.7 Hybrid Retrieval with Reranking Experiment

In the hybrid reranking experiment, we apply cross encoder reranking after hybrid candidate retrieval. The hybrid retriever provides 20 candidates and the reranker chooses the best 10. This is the most complex pipeline currently implemented. This experiment tests whether hybrid recall combined with reranked precision generates better answers than previous implementations at a higher computational cost.

5.5 Benchmarking and Data Collection

All results are stored in a shared CSV file via our project's results logger. Prior to running each notebook, we clear out any rows for that specific implementation so repeated runs overwrite old data rather than produce multiple entries. Each row includes implementation name, scenario ID, question, generated answer, source documents, departmental information from retrieved sources, context information from retrieved sources, number of retrieved documents, top- k settings, indexing time, retrieval time and generation time.

It is crucial to note that there is an important difference between index-

ing time, which represents one-time costs associated with initial document preparation and storage; whereas retrieval time and generation time represent ongoing costs during answer production.

Once all scenario runs have completed, we take the results file to create the manual evaluation worksheet. Using evidence from the DOCX corpus we fill in manual scores and supporting documents. Then our deterministic evaluation script calculates several relevant metrics including retrieval and answer quality and creates two output files per-scenario and summary CSV files. Our visualization notebook then utilizes these preprocessed outputs to create various types of plots including answer-quality, retrieval-quality, scenario-based plots, timing-based plots, and radar-charts for Chapter 6.

5.6 Data Validation

The recorded output is validated as complete prior to analysis. The validation entails verification that there are five rows in each of the implementations, that the generated answers were not null, that there were sources that were retrieved, and that the time fields were available. Additionally, all retrieved contexts are manually examined when developing the truth table (ground-truth sheet), since the quality score of the answer cannot be assumed; it must have some form of evidence based upon a subset of the data within the corpus.

5.7 Summary

This chapter describes the experimental setting and execution for the developed RAG pipelines. It provides information on the software environment, model services, input corpus, notebooks execution order, deployment phases, question types used for scenarios, specific experiment designs for each of the pipelines, log results for each of the pipelines, and validation procedure. The local phase is where the comparison took place for the different retrieval approaches with respect to their performance by comparing them on a local computer.

Chapter 6 – Results and Discussion

6.1 Introduction

This Chapter describes and evaluates the data collected during the execution states (both local and HPC). The evaluation of the RAG-variants implemented locally will be performed with respect to timing, as well as on the quality of the retrieved and given answers, which have been specified in chapters 4 and 5.

6.2 Results of the Local Runs

For the six local versions: (1) an in memory version, (2) a Milvus based version, (3) a Milvus based version which has been optimized (tuned), (4) a dense reranking version, (5) a hybrid-retrieval version, and finally (6) a version where both reranking and hybrid-retrieval are combined, we have the following results:

Table 6.1: Summary of local evaluation results averaged across the five scenarios.

Implementation	Manual	Ret. F1	Index	Retrieve	Generate	Answer len.
Basic	0.80	0.424	17.65	0.067	27.47	147.0
VectorDB	0.60	0.451	35.43	0.104	21.95	104.0
VectorDB tuned	0.60	0.366	36.87	0.103	31.67	120.8
Reranker	0.50	0.345	57.69	1.148	28.31	85.8
Hybrid	0.40	0.279	34.87	0.175	25.52	116.4
Hybrid rerank	0.60	0.405	62.96	2.988	35.79	119.0

As indicated by Table 6.1, the best results for both manual answer scores (answer quality) and retrieval F1 (retrieval quality), were obtained using different pipelines. Thus it is clear that while there was a correlation between these two aspects of overall pipeline performance, they are not synonymous. It may be possible to obtain high-quality retrieved content in terms of relevance, but fail to generate the complete or correct answer due to the lack of detail selection during generation.

6.2.1 Local Timing Comparison

Figure 6.1 illustrates the average execution time for each of the local runs. The simplest in-memory pipeline was able to index faster than all other configurations with an average of 17.65 seconds. The pipelines that utilized a Milvus backend took longer to index since the document chunks and their corresponding embeddings were being stored on an outside vector data base. The reranking and hybrid-reranking pipelines had the longest indexing times (57.69 and 62.96 seconds) as these are configured to perform additional retrieval operations and therefore have a greater number of steps within their setups.

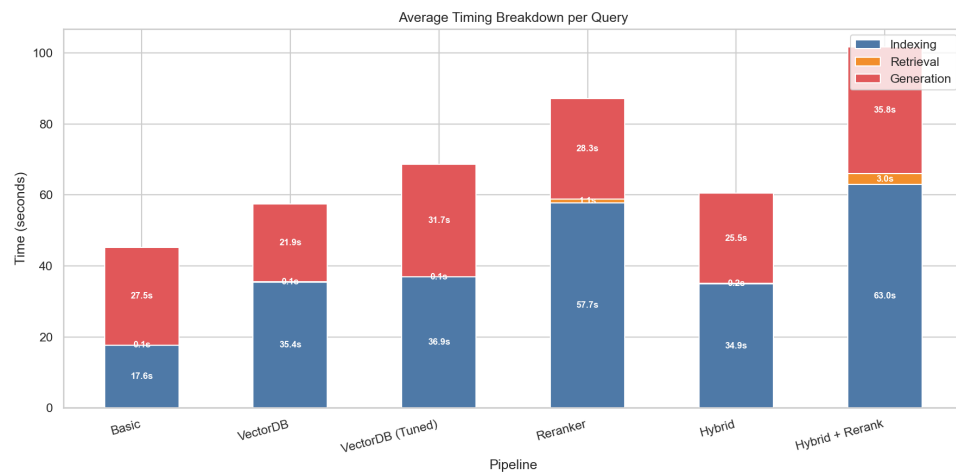


Figure 6.1: Average local timing breakdown for indexing, retrieval, and generation across the implemented RAG variants.

Retrieval times were very short for all three of our query-pipelines (the "basic" pipeline, the "original" vector database pipeline, and the "tuned" vector database pipeline). The Basic pipeline took on average 0.067 seconds, the Original Vector Database Pipeline took on average about 0.104 seconds, and the Tuned Vector Database Pipeline took on average about 0.103 seconds. The use of a Hybrid Retrieval method resulted in longer retrieval times with an average of 0.175 seconds and, as expected, Reranking caused longer retrieval times since another cross encoder pass was made to the Candidate Set. The longest retrieval time was from the Hybrid-Reranking Pipeline with an average of 2.988 seconds.

The Generation Times for each of the implementations were greater than the Retrieval Times for each of the implementations. The Tuned Vector-Database Pipeline had the least amount of Average Generation Time with an average of 21.95 seconds and the Hybrid-Reranking Pipeline had the largest Average Generation Time with an average of 35.79 seconds. This indicates that in this

local environment, the Language Model Generation Step dominated the Per-Query Response Cost. Thus, although retrieval optimization is important, the latency difference was substantially less than that due to Generation Latency, other than when using a Hybrid-Reranking Pipeline.

6.2.2 Local Retrieval and Answer Quality

Figure 6.2 provides an answer-quality comparison. Of the pipelines shown in Figure 6.2, the Basic Pipeline received the highest Manual Score at 0.80 which included 2 *Correct Answers* and 1 *Partially-Correct Answer*. All other pipelines had scores that were lower than the Basic Pipeline. Vector-Database had a score of 0.60 as did Tuned Vector Database and Hybrid-Reranking. Reranking had a score of 0.50 while Hybrid was slightly lower with a score of 0.40.

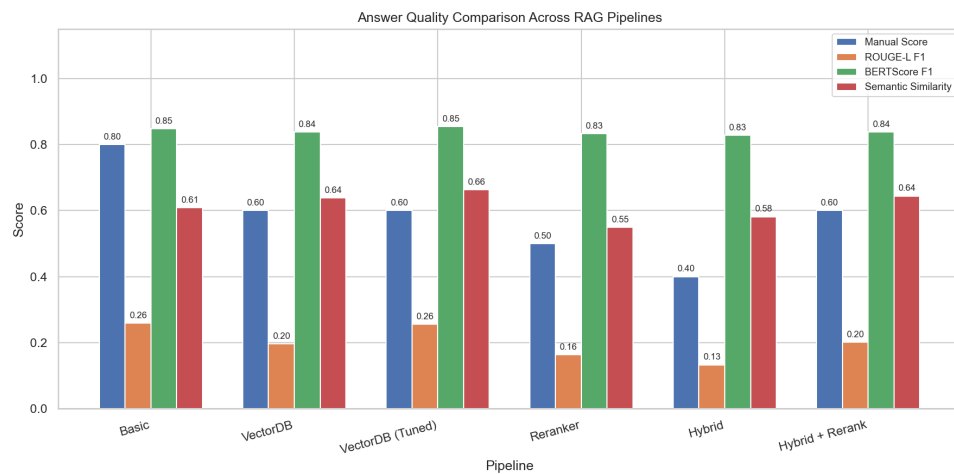


Figure 6.2: Answer-quality metrics for the local RAG variants, including manual score, ROUGE-L, BERTScore, and semantic similarity.

The results from using automatic semantic metrics did not always match up perfectly to the manual scores. For example, although the manually-tuned version of the vector-database pipeline got the highest BERTScore and semantic-similarity (which both reward how much information overlaps with a given reference-answer) it only received a manual-score of 0.60. There are two reasons for this discrepancy. First, semantic-similarity only looks at how much information in your response overlaps with the information in the reference answer. Second, when doing manual-scoring we look to see if you provided enough detail about each operation, or if you left anything out.

As shown in Figure 6.3, the vector-database-pipeline produced the best results for all retrieval-based metrics. It scored an F1 of 0.451, which

represented the highest value for that metric; and it also scored a precision of 0.319, again representing the highest value for that metric. The basic-pipeline scored an F1 of 0.424, and hybrid-reranking scored an F1 of 0.405. All four of those pipelines (basic-pipeline, vector-database-pipeline, tuned-vector-database-pipeline, and hybrid-reranking) scored a perfect 100% on retrieval-recall. The reranker scored 90%, and the hybrid-pipeline scored 70%.

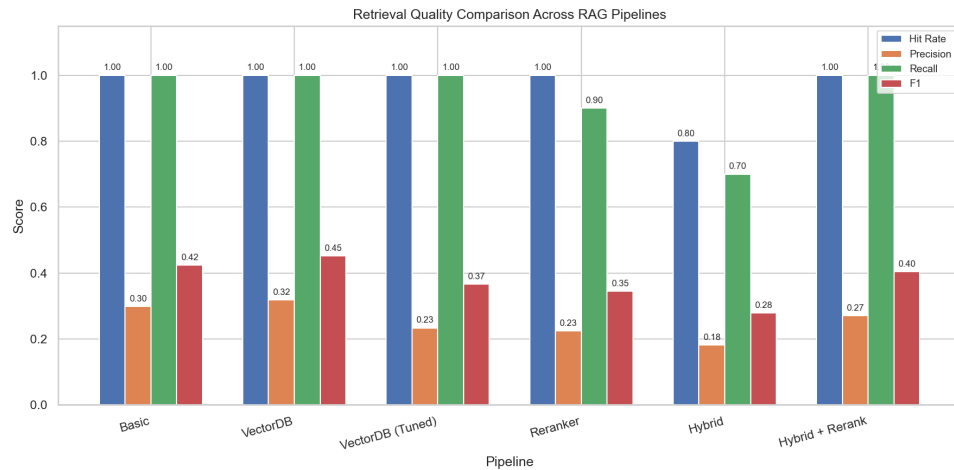


Figure 6.3: Retrieval-quality metrics for the local RAG variants, comparing hit rate, precision, recall, and F1.

The results from this local test indicate that hybrid pipeline performance did not improve overall search quality. Hybrid search is expected to improve performance as a combination of both lexical and semantic cues can enhance search results. However, neither the Milvus hybrid configuration nor the small number of scenarios used in this study provided a significant improvement over the dense-vector database-pipeline.

Hybrid-reranking variant had improved the retrieval-quality in comparison to the hybrid-only, however, there was also a large increase in the time for retrieving results.

Manual score-by-scenario for Figure 6.4 shows that scenario 1, Sev-1 SLA Lookup, was the least difficult (average score = .92). Scenario 3 and 5 were the most difficult (both averaged .42) as they required cross-functionally reasoning of onboarding, IT, Logistics, Customer Success, Product and Commercial. Low scores suggest that searching for the correct documents is insufficient if the user needs information regarding multiple operationally dependent processes.

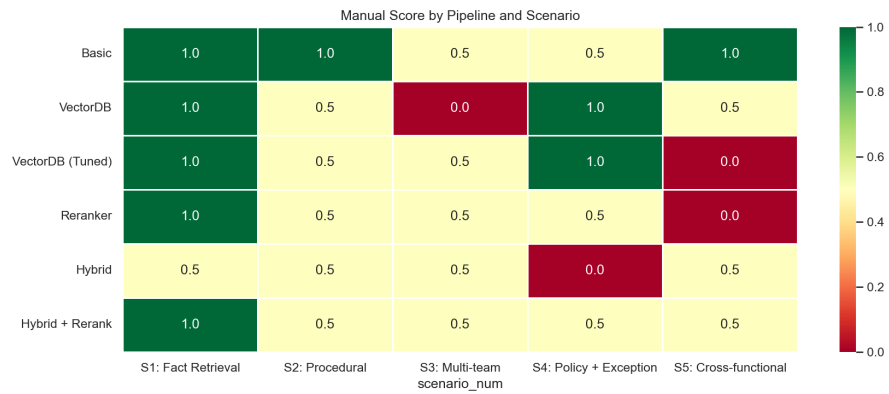


Figure 6.4: Manual answer score by implementation and scenario. Darker cells indicate stronger manually evaluated answers.

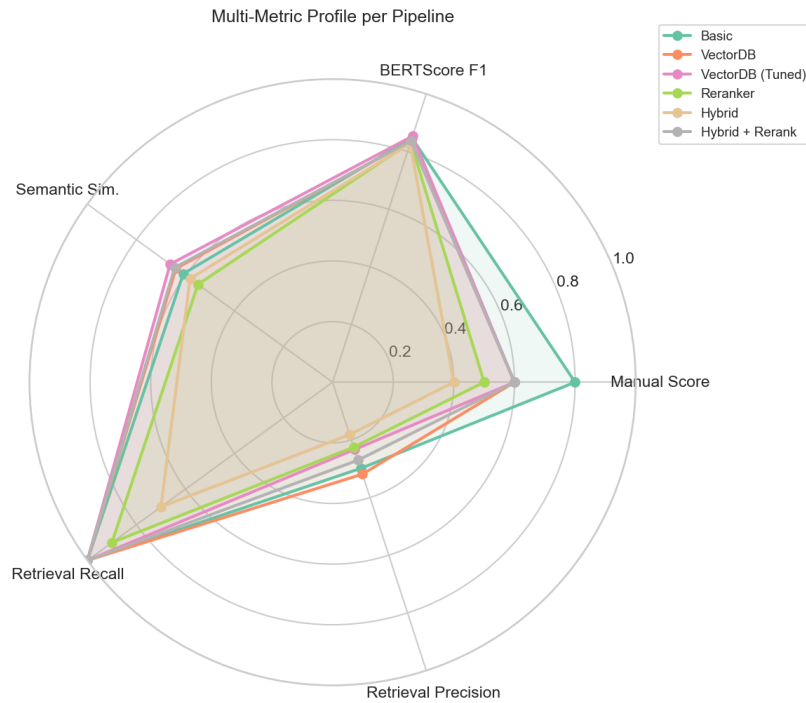


Figure 6.5: Multi-metric profile of the local RAG pipelines, comparing manual score, BERTScore, semantic similarity, retrieval recall, and retrieval precision.

6.3 HPC Deployment Results

6.4 Local versus HPC Comparison

6.5 Effectiveness of Retrieval-Level Optimization Techniques

Local results showed mixed effectiveness for Retrieval-Level Optimization Techniques. Dense retrieval using milvus-backed vectors enhanced local F1 scores comparing to in-memory baseline but did not improve manual answer score. This suggests that storing vectors persistently provides a stronger foundation for retrieving data without always enhancing generative quality.

The tuned vector-database run yielded better ROUGE-1, BERTScore and semantic similarity than the original vector database run; however it also decreased local retrieval F1 score. Larger chunks may have given the generator more complete context while also making source level retrieval less accurate. This additional support for chunk size being an experimental parameter with meaning, rather than just a technical process choice.

Reranking did not enhance local manual score in this particular experiment and in fact caused average latency of retrieval to go from sub second range to over one second on average. The hybrid-reranking pipeline was better than hybrid retrieval alone and equivalent to the manual score of vector database variant, but had longest times for both retrieval and generation. Thus, there is no doubt as to the cost associated with reranking and its benefit in terms of quality was limited for this corpus and scenario set.

6.6 Comparison with Literature

The results confirm the body of literature’s observations about how RAG performance varies with both retrieval quality and generation characteristics. For example, the research demonstrates that dense retrieval is very successful for a controlled Enterprise Corpus (especially when the predicted evidence exists in documents describing policies, handbooks etc. with the same semantic content). At the same time, one can observe that having a large number of retrievals does not always result in an entirely accurate answer.

Even though some evidence was found, some generated answers failed to include at least one required owner, escalation trigger, or accountability condition (e.g., even if there were many sources which contained information about them).

The reranked results are consistent with the expected computational cost/latency trade-offs documented in past literature. While cross-encoders may significantly improve candidate ordering, they add significant additional processing beyond what occurred during the first retrieval. Given the constraints of the experimental design (i.e., the size of the source corpus, the length of candidate lists, and the number of different evaluation test cases), expectations include a proportionally greater increase in manual quality evaluations of candidates based upon their use of additional computation.

Hybrid retrieval results were less strong than anticipated. As noted earlier, hybrid retrieval has been presented as a way to combine the benefits of using exact keyword matches along with retrieving relevant documents from a larger source document collection. Nevertheless, achieving the desired performance gains through hybrid retrieval requires careful tuning of parameters including (but certainly not limited to): configuration settings used for hybrid retrieval, source document collection structures, sparsity levels for vectors representing documents, and query types. Therefore, a belief is these local findings provide sufficient justification for cautionary conclusions. Hybrid retrieval should be carefully tuned, evaluated and compared against dense retrieval before being selected for deployment as a replacement for dense retrieval.

6.7 Trade-Offs and Limitations

While there are many theoretical trade-offs between this approach and others (e.g., what trade-offs will occur as the number of documents grows), the most relevant trade-off for the immediate future appears to be that between *infrastructure-realism* and *simplicity*. In other words, while the baseline in-memory pipeline produced both the best manual score and the shortest indexing times, it is certainly less like an actual deployable application. On the other hand, the vector database pipeline took longer to index (i.e. to set up) than the baseline in memory pipeline, but it produced better retrieval results (i.e. stronger F1 values) and a more reasonable storage basis.

There was another trade-off. Namely one between retrieval quality and

query-time. As would be expected, reranking (and hybrid reranking) made substantial increases in the time taken to retrieve answers. However, when a comparison was made between hybrid reranking and hybrid retrieval alone, hybrid reranking resulted in slightly better-quality retrievals, even though it did so at a cost of greater query latency. Given these two observations, the trade-off does appear worthwhile to pursue further, however, whether or not to incur this penalty depends on how much value can be placed upon the extra retrieval accuracy.

There are several important limitations of this evaluation. (i) First, since the corpus is artificial (synthetic), and since the scenario-set contains only five questions, the results have little relevance beyond the specific context used here. (ii) Second, because each candidate was evaluated using only a very simple rubric to manually evaluate their output, the scores do not reflect any subtle differences in performance. (iii) Third, since the evaluation of each candidate was performed locally, results depend heavily on the runtime environment of Ollama, the models' initial warmup time, the presence/absence of background processes running during testing, and the current state of the Milvus service.

6.8 Summary

Chapter 7 – Conclusion and Future Work

7.1 Introduction

7.2 Summary of Findings

7.3 Key Contributions

The main contributions of the thesis are expected to include:

- a focused explanation of LLM application approaches, including prompting, RAG, fine-tuning, and training from scratch;
- a practical implementation of five comparable RAG pipelines;
- a local benchmark comparison of those pipelines;
- a reproducible workflow designed so the same files can be executed on HPC;
- a combined timing and output-quality evaluation framework; and
- a final local-versus-HPC comparison

7.4 Limitations

7.5 Future Work

7.6 Final Remarks

Bibliography

- [1] J. Alammam, “The illustrated transformer,” <https://jalammar.github.io/illustrated-transformer/>, 2018.
- [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [3] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems*, 2020. [Online]. Available: <https://arxiv.org/abs/2005.14165>
- [4] J. Kaplan, S. McCandlish, T. Henighan *et al.*, “Scaling laws for neural language models,” *arXiv preprint arXiv:2001.08361*, 2020. [Online]. Available: <https://arxiv.org/abs/2001.08361>
- [5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems*, 2020. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [6] G. Izacard and E. Grave, “Leveraging passage retrieval with generative models for open domain question answering,” in *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, 2021, pp. 874–880. [Online]. Available: <https://arxiv.org/abs/2007.01282>
- [7] S. Borgeaud, A. Mensch, J. Hoffmann *et al.*, “Improving language models by retrieving from trillions of tokens,” in *International Conference on Machine Learning*, 2022. [Online]. Available: <https://arxiv.org/abs/2112.04426>
- [8] G. M. Kurtzer, V. Sochat, and M. W. Bauer, “Singularity: Scientific containers for mobility of compute,” *PLOS ONE*, vol. 12, no. 5, 2017. [Online]. Available: <https://doi.org/10.1371/journal.pone.0177459>

- [9] A. B. Yoo, M. A. Jette, and M. Grondona, “Slurm: Simple linux utility for resource management,” in *Job Scheduling Strategies for Parallel Processing*. Springer, 2003. [Online]. Available: https://doi.org/10.1007/10968987_3
- [10] J. Dongarra, P. Beckman, T. Moore, P. Aerts *et al.*, “High-performance computing: Challenges and opportunities,” *International Journal of High Performance Computing Applications*, 2020. [Online]. Available: <https://doi.org/10.1177/1094342020957393>
- [11] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: Bm25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009. [Online]. Available: <https://doi.org/10.1561/15000000019>
- [12] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020, pp. 6769–6781. [Online]. Available: <https://aclanthology.org/2020.emnlp-main.550/>
- [13] J. Johnson, M. Douze, and H. Jegou, “Billion-scale similarity search with gpus,” *IEEE Transactions on Big Data*, 2017. [Online]. Available: <https://ieeexplore.ieee.org/document/8733051>
- [14] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020. [Online]. Available: <https://ieeexplore.ieee.org/document/8626655>
- [15] R. Nogueira and K. Cho, “Passage re-ranking with bert,” in *arXiv preprint arXiv:1901.04085*, 2019. [Online]. Available: <https://arxiv.org/abs/1901.04085>
- [16] O. Khattab and M. Zaharia, “Colbert: Efficient and effective passage search via contextualized late interaction over bert,” in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020, pp. 39–48. [Online]. Available: <https://dl.acm.org/doi/10.1145/3397271.3401075>
- [17] J. Frankle and M. Carbin, “The lottery ticket hypothesis: Finding sparse, trainable neural networks,” in *Proceedings of the International*

- Conference on Learning Representations*, 2019. [Online]. Available: <https://arxiv.org/abs/1803.03635>
- [18] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” *arXiv preprint arXiv:2106.09685*, 2022. [Online]. Available: <https://arxiv.org/abs/2106.09685>
- [19] J. Pfeiffer, A. Kamath, A. Ruckle, K. Cho, and I. Gurevych, “Adapterfusion: Non-destructive task composition for transfer learning,” in *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, 2021, pp. 487–503. [Online]. Available: <https://aclanthology.org/2021.eacl-main.39/>
- [20] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, “Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing,” *arXiv preprint arXiv:2107.13586*, 2023. [Online]. Available: <https://arxiv.org/abs/2107.13586>
- [21] C.-Y. Lin, “ROUGE: A package for automatic evaluation of summaries,” in *Text Summarization Branches Out*. Association for Computational Linguistics, 2004, pp. 74–81. [Online]. Available: <https://aclanthology.org/W04-1013/>
- [22] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “BERTScore: Evaluating text generation with BERT,” in *International Conference on Learning Representations*, 2020. [Online]. Available: <https://openreview.net/forum?id=SkeHuCVFDr>
- [23] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019, pp. 3982–3992. [Online]. Available: <https://aclanthology.org/D19-1410/>
- [24] W. Zhang, C. Li, and A. Kumar, “A survey on machine learning system design: Performance, scalability, and efficiency,” *ACM Computing Surveys*, 2021. [Online]. Available: <https://doi.org/10.1145/3439723>
- [25] P. Liang, R. Bommasani *et al.*, “Holistic evaluation of language models,” *arXiv preprint arXiv:2211.09110*, 2022. [Online]. Available: <https://arxiv.org/abs/2211.09110>

- [26] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “Ragas: Automated evaluation of retrieval augmented generation,” *EACL 2024 - 18th Conference of the European Chapter of the Association for Computational Linguistics, Proceedings of System Demonstrations*, pp. 150–158, 2024. [Online]. Available: <https://aclanthology.org/2024.eacl-demo.16/>
- [27] D. Narayanan, K. Santhanam, A. Phanishayee, and M. Zaharia, “Efficient large-scale language model serving on cloud platforms,” *arXiv preprint arXiv:2109.00053*, 2021. [Online]. Available: <https://arxiv.org/abs/2109.00053>
- [28] M. Shoeybi, M. Patwary, R. Puri, P. Le *et al.*, “Megatron-lm: Training multi-billion parameter language models using model parallelism,” *arXiv preprint arXiv:1909.08053*, 2019. [Online]. Available: <https://arxiv.org/abs/1909.08053>
- [29] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, “Zero: Memory optimization towards training trillion parameter models,” *arXiv preprint arXiv:1910.02054*, 2020. [Online]. Available: <https://arxiv.org/abs/1910.02054>
- [30] A. Abdolrashidi, P. Mattson, C.-J. Wu, Y. Cheng *et al.*, “Large-scale evaluation of deep learning workloads on hpc platforms,” in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 883–895.
- [31] W. Kwon, B. Y. Li, Y. J. Lee, D. Patterson, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” *arXiv preprint arXiv:2309.06180*, 2023. [Online]. Available: <https://arxiv.org/abs/2309.06180>
- [32] T. Dettmers, M. Lewis, S. Shleifer, and L. Zettlemoyer, “Llm.int8(): 8-bit matrix multiplication for transformers at scale,” *arXiv preprint arXiv:2208.07339*, 2022. [Online]. Available: <https://arxiv.org/abs/2208.07339>
- [33] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “Distilbert, a distilled version of bert: Smaller, faster, cheaper and lighter,” in *Proceedings of the 5th Workshop on Energy Efficient Machine Learning and Cognitive Computing*, 2019. [Online]. Available: <https://arxiv.org/abs/1910.01108>

- [34] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, “Flashattention: Fast and memory-efficient exact attention with io-awareness,” in *Advances in Neural Information Processing Systems*, 2022. [Online]. Available: <https://arxiv.org/abs/2205.14135>
- [35] H. Chase, “Langchain: Building applications with large language models,” *Technical Report*, 2023. [Online]. Available: <https://arxiv.org/abs/2307.11536>
- [36] J. Liu, “Llamaindex: A data framework for llm applications,” *Technical Report*, 2023. [Online]. Available: <https://arxiv.org/abs/2308.07862>
- [37] V. J. Reddi, C. Cheng *et al.*, “Mlperf inference benchmark,” in *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture*, 2020. [Online]. Available: <https://arxiv.org/abs/1911.02549>

Appendices

Appendix A: Reproducibility and Execution Guide

This appendix provides supporting information for reproducing the experimental workflow used in this thesis. The experiments were implemented as Jupyter notebooks and executed locally using the same document corpus, model configuration, scenario questions, logging format, and evaluation scripts.

The main notebooks used in the experiments were:

- `rag_basic.ipynb`
- `rag_vectordb.ipynb`
- `rag_vectordb_tuned.ipynb`
- `rag_reranker.ipynb`
- `rag_hybrid.ipynb`
- `rag_hybrid_&_rerank.ipynb`

The local execution environment used Ollama for model serving and Milvus for the vector-database and hybrid-retrieval pipelines. The generation model was `llama3.1`, while the embedding model was `mxbai-embed-large`. Each notebook followed the same general workflow: loading the document corpus, splitting documents into chunks, indexing or storing embeddings, executing the scenario questions, generating answers, and saving results to the shared results file.

This appendix is intended to support reproducibility by documenting the practical execution order and implementation files used to generate the experimental outputs discussed in the main thesis.

Appendix B: Evaluation Scenarios and Scoring

This appendix lists the scenario questions used to evaluate the implemented RAG pipelines. The same questions were used across all pipeline variants to ensure that differences in retrieval quality, answer quality, and execution time were caused by the retrieval strategy rather than by changes in the input task.

1. What is the first-response SLA for a Sev-1 support incident?

This scenario tests direct factual retrieval from a single relevant document.

2. What steps must be completed before a new employee is considered ready for day one at OrchestrAI?

This scenario tests whether the system can retrieve and summarize a procedural checklist.

3. A new engineer starts on Monday, but their laptop has not arrived and their required access is still pending. Which teams are involved and what should happen next?

This scenario tests cross-document retrieval involving HR, IT, and access-related responsibilities.

4. Can OrchestrAI make an urgent hardware purchase without following the normal PO workflow if a warehouse outage is at risk?

This scenario tests policy retrieval and exception-based reasoning.

5. A customer renewal is at risk because support cases are recurring, a requested feature is still unavailable, and replacement hardware is delayed. What should OrchestrAI do next, and which teams must be involved?

This scenario tests multi-hop retrieval across support, product, logistics, and sales-related information.

Manual answer quality was scored using three values: 1.0 for a correct answer, 0.5 for a partially correct answer, and 0.0 for an incorrect or unsupported answer. Retrieval quality was evaluated using whether the expected supporting documents appeared in the retrieved sources, together with precision, recall, and F1. Automated answer-quality metrics included ROUGE-L, BERTScore, sentence-embedding semantic similarity, and answer length.

Manuals

Installation Manual

This manual describes the steps required to install and prepare the local environment used for the RAG experiments.

System Requirements

The project was executed locally using Python, Jupyter Notebook, Ollama, and Milvus. The experiments require enough memory and storage to load the document corpus, generate embeddings, and execute the RAG pipelines.

Python Environment

A Python virtual environment should be created before installing the project dependencies. After activating the environment, the required packages can be installed from the project requirements file.

Model Setup

The experiments use Ollama for local model serving. The generation model used in the thesis is llama3.1, and the embedding model is mxbai-embed-large. Both models must be available before running the notebooks.

Milvus Setup

The vector-database, tuned-vector-database, hybrid, reranking, and hybrid-reranking experiments require a local Milvus service. The service is started using the project script `scripts/standalone_embed.sh`.

Project Data

The OrchestrAI document corpus must remain in the expected project directory structure so that each notebook can load the same input files.

User Manual

This manual explains how to run the implemented RAG pipelines and generate the experimental outputs.

Running the Notebooks

The notebooks should be executed in the following order:

1. `rag_basic.ipynb`
2. `rag_vectordb.ipynb`
3. `rag_vectordb_tuned.ipynb`
4. `rag_reranker.ipynb`
5. `rag_hybrid.ipynb`
6. `rag_hybrid_&_rerank.ipynb`

Each notebook loads the same document corpus, executes the same five scenario questions, records retrieved sources and generated answers, and writes timing information to the shared results file.

Evaluation Workflow

After the notebooks have been executed, the generated results are used to create the manual evaluation sheet. Manual correctness scores are then added, and the deterministic evaluation script calculates retrieval and answer-quality metrics.

Generating Figures

The visualization notebook reads the processed results and generates figures for answer quality, retrieval quality, timing breakdown, scenario comparison, and radar-chart summaries.

Interpreting Outputs

The most important outputs are the generated answers, retrieved sources, timing fields, manual correctness scores, retrieval metrics, and automated answer-quality metrics. These outputs are used in the results chapter to compare the six RAG pipeline variants.